FINÁLNÍ PROJEKT
č. 2



Autor: Jan Juřina
Datum: 12. 11. 2026

OBSAH

ZADÁNÍ **3**
TESTOVACÍ SCÉNÁŘE **4**
EXEKUCE TESTŮ **100**
BUG REPORT **129**

ZADÁNÍ

Cílem finálního projektu je otestovat funkčnost aplikace, která slouží k manipulaci s daty o studentech. Aplikace má rozhraní REST-API, které umožňuje vytvoření, smazání a získání dat.

Přístupové údaje:

Databáze	**Default scheme**: students **Host**: test-app.engeto.cz **Port**: 5433
REST-API REST-API dokumentace	https://test-app.engeto.cz/students https://test-app.engeto.cz/docs

Poznámky:

Nezapomeňte, že v IT se data musí někde uložit a poté získat. Proto ověřte, že data jsou správně uložena a získávána z databáze.
Nezapomeňte do testovacích scénářů uvést testovací data, očekávaný výsledek včetně těla odpovědi a stavových kódů.

TESTOVACÍ SCÉNÁŘE

TC_GET_001 – Načtení existujícího studenta

Cíl: Načtení existujícího studenta.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt s `id: 376`; `firstName`, `lastName`, `email`, `isEuCitizen` (boolean) a `numberOfFailedStudies` (integer) se musí přesně shodovat s výsledkem SQL SELECT pro ID 376.
SQL dotaz v DBeaveru:

```
SELECT *
FROM public.students
WHERE id = 376;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_002 – Neexistující student

Cíl: Neexistující student.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/999999999
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru:

```
SELECT *
FROM public.students
WHERE id = 999999999;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_003 – Neplatný datový typ ID

Cíl: Neplatný datový typ ID.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/abc.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/abc
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné; ID v URL = abc.
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_004 – Chybějící autorizace

Cíl: Chybějící autorizace.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/376.
2. Nastavit autorizaci (Bez Authorization hlavičky (No Auth).); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/376
Autorizace: Bez Authorization hlavičky (No Auth).
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 376;
Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_005 – Neplatný token

Cíl: Neplatný token.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/376.
2. Nastavit autorizaci (Bearer Token: neplatný testovací token.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/376
Autorizace: Bearer Token: neplatný testovací token.
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 376;
Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_006 – Záporné ID studenta

Cíl: Záporné ID studenta.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/-1.
2. Nastavit autorizaci (Bearer Token: {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/-1
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 404 Not Found nebo 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru:
```
SELECT *
FROM public.students
WHERE id = -1;
```
Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_007 – Ověření PSČ přes GET

Cíl: Ověření PSČ přes GET.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/382.
2. Nastavit autorizaci (Bearer Token: {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 382,"firstName": "TestPSCPlatne","lastName": "STUDENT","email": "test.psc.platne.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT id, first_name, zip_code
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_008 – Ověření nově vytvořeného studenta

Cíl: Ověření nově vytvořeného studenta.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/398.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/398
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt s `id: 398` a `email: "TEST.VELKA.PISMENA.20261029@EXAMPLE.COM"`; ostatní atributy se musí shodovat se záznamem ID 398 v databázi.
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 398;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_009 – Přístup ke studentovi pomocí desetinného ID

Cíl: Přístup ke studentovi pomocí desetinného ID.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.5.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403.5
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_010 – ID s nulami na začátku

Cíl: ID s nulami na začátku.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/000403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/000403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_011 – Překročení maximálního rozsahu ID

Cíl: Překročení maximálního rozsahu ID.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/2147483648.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit SQL dotaz pro ID 2147483648 a ověřit, že nevrátí žádného studenta. GET nemění data.

Metoda a URL: GET {{base_url}}/students/2147483648
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name, email
FROM public.students
WHERE id = 2147483648;
```

Očekávaný stav databáze: SQL SELECT pro ID 2147483648 nevrátí žádný řádek. GET nevytváří ani neupravuje záznamy.

TC_GET_012 – ID na maximální hranici integeru

Cíl: ID na maximální hranici integeru.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/2147483647.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/2147483647
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE id = 2147483647;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_013 – ID obsahující pouze mezery

Cíl: ID obsahující pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/%20%20%20.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/%20%20%20
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_014 – ID s kladným znaménkem

Cíl: ID s kladným znaménkem.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/%2B403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/%2B403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email,
    is_eu_citizen,
    number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_015 – ID s desetinnou nulou

Cíl: ID s desetinnou nulou.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.0.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403.0
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity při striktní validaci ID; 200 OK pouze pokud je výslovně povolena normalizace zápisu 403.0 na 403.
Očekávané tělo odpovědi: Při 422 JSON s chybou VALIDATION_ERROR pro parametr student_id; při 200 JSON studenta s id = 403 a hodnotami odpovídajícími databázi.
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_016 – ID ve vědeckém zápisu

Cíl: ID ve vědeckém zápisu.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/4.03e2.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/4.03e2
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK nebo 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_017 – SQL injection v identifikátoru studenta

Cíl: SQL injection v identifikátoru studenta.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403%20OR%201%3D1.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403%20OR%201%3D1
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_018 – HTML/JavaScript v identifikátoru

Cíl: HTML/JavaScript v identifikátoru.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/%3Cscript%3Ealert.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/%3Cscript%3Ealert(1)%3C%2Fscript%3E
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity pro neplatný celočíselný identifikátor; 404 Not Found je možné při odmítnutí cesty routerem.
Očekávané tělo odpovědi: Při 422 {"code":"VALIDATION_ERROR"}; při odmítnutí na úrovni routeru (404) {"detail":"Not Found","code":"NOT_FOUND"}.
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email,
    is_eu_citizen,
    number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_019 – Neplatný Bearer token

Cíl: Neplatný Bearer token.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** neplatny.token.12345.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403
Autorizace: Bearer Token s hodnotou `neplatny.token.12345` (záměrně neplatný; nepoužívat {{token}}).
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized s INVALID_TOKEN; odmítnutí 403 Forbidden se hodnotí jako odchylka od očekávání, nikoli zpřístupnění dat.
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email,
    is_eu_citizen,
    number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_020 – Neplatný token ve správném formátu JWT

Cíl: Neplatný token ve správném formátu JWT.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** neplatný testovací token.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403
Autorizace: Bearer Token: neplatný testovací token.
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email,
    is_eu_citizen,
    number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_021 – Přístup s tokenem bez schématu Bearer

Cíl: Přístup s tokenem bez schématu Bearer.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.
2. **V Postmanu zvolit No Auth** a ručně přidat hlavičku Authorization: {{token}} bez předpony Bearer. Použít ID uvedené v URL; tělo požadavku neposílat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403
Autorizace: No Auth v Postmanu; ruční hlavička Authorization: {{token}} (bez předpony Bearer).
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 403 Forbidden nebo 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_022 – Nesprávné autorizační schéma Basic

Cíl: Nesprávné autorizační schéma Basic.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.
2. V Postmanu nastavit autorizační schéma Basic (nikoliv Bearer) a odeslat požadavek.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403
Autorizace: schéma Basic v hlavičce Authorization, nikoli Bearer.
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 401 Unauthorized nebo 403 Forbidden
Očekávané tělo odpovědi: {"detail": "Invalid authentication credentials","code": "FORBIDDEN"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_GET_023 – Platný token s nesprávně napsaným schématem

Cíl: Platný token s nesprávně napsaným schématem.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/students/403.
2. V Postmanu nastavit režim No Auth, ale ručně přidat hlavičku Authorization: bearer {{token}} (malé písmeno b).
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; porovnat vrácené hodnoty s API a potvrdit, že GET neprovádí zápis.

Metoda a URL: GET {{base_url}}/students/403
Autorizace: No Auth v Postmanu; ruční hlavička Authorization: bearer {{token}} (malé b ve schématu).
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    email,
    is_eu_citizen,
    number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Bez změny v DB; údaje v API odpovídají hodnotám z SELECT.

TC_POST_001 – Vytvoření studenta s platnými údaji

Cíl: Vytvoření studenta s platnými údaji.
Postup testu:
1. V Postmanu připravit požadavek POST https://test-app.engeto.cz/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST https://test-app.engeto.cz/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "TestJan",
  "lastName": "Novak",
  "email": "testjan.novak.20261029@example.com",
  "isEuCitizen": true,
  "zipCode": "27201",
  "numberOfFailedStudies": 0
}
```

Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 376,"firstName": "TestJan","lastName": "Novak","email": "testjan.novak.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 376;
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_002 – Vytvoření studenta s prázdným JSON

Cíl: Vytvoření studenta s prázdným JSON.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: {}
Očekávaný HTTP stav: 201 Created (všechna pole jsou dle API nepovinná)
Očekávané tělo odpovědi: {"id":377,"firstName":null,"lastName":null,"email":null,"isEuCitizen":false,"numberOfFailedStudies":0}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  zip_code,
  number_of_failed_studies
FROM public.students
WHERE id = 377;
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_003 – Neplatný počet neúspěšných studií

Cíl: Neplatný počet neúspěšných studií.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "TestNegativni",
  "lastName": "Student",
  "email": "test.negativni.20261029@example.com",
  "isEuCitizen": true,
  "zipCode": "27201",
  "numberOfFailedStudies": -1
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE email = 'test.negativni.20261029@example.com';
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_004 – Neplatný datový typ

Cíl: Neplatný datový typ.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "TestDatovyTyp",
  "lastName": "Student",
  "email": "test.datovy.typ.20261029@example.com",
  "isEuCitizen": true,
  "zipCode": "27201",
  "numberOfFailedStudies": "abc"
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE email = 'test.datovy.typ.20261029@example.com';
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_005 – Příliš dlouhé jméno

Cíl: Příliš dlouhé jméno.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName":
"AAAA
```

TC_POST_006 – Jméno o délce přesně 255 znaků

Cíl: Jméno o délce přesně 255 znaků.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName":
"AAAA
```

TC_POST_008 – Vytvoření studenta bez autorizace

Cíl: Vytvoření studenta bez autorizace.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bez Authorization hlavičky (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bez Authorization hlavičky (No Auth).
Testovací data / JSON:

```
{
  "firstName": "TestBez",
  "lastName": "Autorizace",
  "email": "test.bez.autorizace.20261029@example.com",
  "isEuCitizen": true,
  "numberOfFailedStudies": 0
}
```

Očekávaný HTTP stav: 401 Unauthorized nebo 403 Forbidden – bez tokenu nesmí být student vytvořen
Očekávané tělo odpovědi: Při 403 {"detail":"Not authenticated","code":"FORBIDDEN"}; při 401 odpovídající autorizační chyba. V obou případech nesmí dojít k zápisu ani smazání studenta.
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE email = 'test.bez.autorizace.20261029@example.com';
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_009 – Duplicitní e-mail

Cíl: Duplicitní e-mail.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "TestDuplicita",
  "lastName": "Email",
  "email": "testjan.novak.20261029@example.com",
  "isEuCitizen": true,
  "zipCode": "27201",
  "numberOfFailedStudies": 0
}
```

Očekávaný HTTP stav: 400 Bad Request – duplicitní e-mail
Očekávané tělo odpovědi: {"detail":"Student with the same e-mail already exists","code":"STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE email = 'testjan.novak.20261029@example.com';
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_010 – Neplatný datový typ boolean

Cíl: Neplatný datový typ boolean.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestBoolean",
  "lastName": "Neplatny",
  "email": "test.boolean.20261029@example.com",
  "isEuCitizen": "abc",
  "zipCode": "27201",
  "numberOfFailedStudies": 0
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean, unable to interpret input","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE email = 'test.boolean.20261029@example.com';
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_011 – Neplatné PSČ

Cíl: Neplatné PSČ.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestPSC",
  "lastName": "Neplatne",
  "email": "test.psc.20261029@example.com",
  "isEuCitizen": true,
  "zipCode": "ABCDE",
  "numberOfFailedStudies": 0
}
```
Očekávaný HTTP stav: 201 Created při ignorování či nevalidování zipCode; jinak 422 Unprocessable Entity (dle specifikace PSČ)
Očekávané tělo odpovědi: Při 201: JSON nového studenta (v testu ID 380), `firstName="TestPSC"`, `lastName="Neplatne"`; při 422: `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru:
```
SELECT *
FROM public.students
WHERE email = 'test.boolean.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_012 – Nepovolené pole v JSON

Cíl: Nepovolené pole v JSON.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestExtra",
 "lastName": "Pole",
 "email": "test.extra.20261029@example.com",
 "isEuCitizen": true,
 "numberOfFailedStudies": 0,
 "favoriteColor": "blue"
}
```

Očekávaný HTTP stav: 201 Created, pokud API ignoruje neznámé pole; nebo 422 Unprocessable Entity, pokud zakazuje dodatečná pole
Očekávané tělo odpovědi: Při 201: JSON nového studenta s přijatými standardními poli; nepovolené pole se do modelu nepropíše. Při 422: JSON `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru:

```
SELECT *
FROM public.students
WHERE email = 'test.extra.20261029@example.com';
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_013 – Uložení platného PSČ při vytvoření

Cíl: Uložení platného PSČ při vytvoření.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestPSCPlatne",
 "lastName": "Student",
 "email": "test.psc.platne.20261029@example.com",
 "isEuCitizen": true,
 "zipCode": "27201",
 "numberOfFailedStudies": 0
}
```

Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 382,"firstName": "TestPSCPlatne","lastName": "Student","email": "test.psc.platne.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, email, zip_code
FROM public.students
WHERE email = 'test.psc.platne.20261029@example.com';
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_014 – Neplatný datový typ jména firstName

Cíl: Neplatný datový typ jména firstName.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": 12345,
 "lastName": "TestTyp",
 "email": "test.typ.20261029@example.com"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string", "code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE email = 'test.typ.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_015 – Neplatný typ příjmení

Cíl: Neplatný typ příjmení.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestTypPrijmeni",
 "lastName": 12345,
 "email": "test.typ.prijmeni.20261029@example.com"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> lastName: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE email = 'test.typ.prijmeni.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_016 – Neplatný typ e-mailu

Cíl: Neplatný typ e-mailu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestEmailTyp",
 "lastName": "CiselnyEmail",
 "email": 12345
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> email: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email
FROM public.students
WHERE first_name = 'TestEmailTyp'
 AND last_name = 'CiselnyEmail';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_017 – E-mail delší než 255 znaků

Cíl: E-mail delší než 255 znaků.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestDlouhyEmail",
 "lastName": "Limit256",
 "email":
"aaaa
```

TC_POST_018 – E-mail přesně 255 znaků

Cíl: E-mail přesně 255 znaků.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestEmail255",
 "lastName": "Limit255",
 "email":
"bbbb
```

TC_POST_020 – Mezery na začátku a konci jména

Cíl: Mezery na začátku a konci jména.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "  TestMezery  ",
 "lastName": "  Novak  ",
 "email": "test.mezery.20261029@example.com"
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 389,"firstName": " TestMezery ","lastName": " NOVAK ","email": "test.mezery.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  '[' || first_name || ']' AS jmeno,
  '[' || last_name || ']' AS prijmeni,
  LENGTH(first_name) AS delka_jmena,
  LENGTH(last_name) AS delka_prijmeni
FROM public.students
WHERE email = 'test.mezery.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_021 – Jméno obsahující pouze mezery

Cíl: Jméno obsahující pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "   ",
 "lastName": "TestPouzeMezery",
 "email": "test.pouze.mezery.20261029@example.com"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> firstName: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  '[' || first_name || ']' AS jmeno,
  LENGTH(first_name) AS delka_jmena,
  last_name,
  email
FROM public.students
WHERE email = 'test.pouze.mezery.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_022 – Jméno s diakritikou

Cíl: Jméno s diakritikou.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "Jiří",
  "lastName": "Dvořáková",
  "email": "test.diakritika.20261029@example.com"
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 391,"firstName": "Jiří","lastName": "DVOŘÁKOVÁ","email": "test.diakritika.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  LENGTH(first_name) AS delka_jmena,
  LENGTH(last_name) AS delka_prijmeni
FROM public.students
WHERE email = 'test.diakritika.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_023 – Záporné ID v těle požadavku

Cíl: Záporné ID v těle požadavku.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "id": -100,
  "firstName": "TestVlastniID",
  "lastName": "ZaporneID",
  "email": "test.zaporne.id.20261029@example.com"
}
```
Očekávaný HTTP stav: 201 Created při ignorování klientem dodaného id; případně 422 Unprocessable Entity při zakázaném id
Očekávané tělo odpovědi: Při 201: JSON nového studenta se serverem přiděleným kladným `id` (nikoliv -100). Při 422: JSON s `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE email = 'test.zaporne.id.20261029@example.com'
   OR id = -100;
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_024 – `numberOfFailedStudies` jako desetinné číslo

Cíl: `numberOfFailedStudies` jako desetinné číslo.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestDesetinneCislo",
 "lastName": "Studium",
 "email": "test.desetinne.studium.20261029@example.com",
 "numberOfFailedStudies": 1.5
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer, got a number with a fractional part","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  number_of_failed_studies
FROM public.students
WHERE email = 'test.desetinne.studium.20261029@example.com';
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_025 – Boolean místo počtu neúspěšných studií

Cíl: Boolean místo počtu neúspěšných studií.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestBooleanStudium",
 "lastName": "Boolean",
 "email": "test.boolean.studium.20261029@example.com",
 "numberOfFailedStudies": true
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email,
  number_of_failed_studies
FROM public.students
WHERE email = 'test.boolean.studium.20261029@example.com';
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_026 – Boolean `false` místo počtu studií

Cíl: Boolean `false` místo počtu studií.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestBooleanFalse",
 "lastName": "Studium",
 "email": "test.boolean.false.20261029@example.com",
 "numberOfFailedStudies": false
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE email = 'test.boolean.false.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_027 – Číslo zadané jako text

Cíl: Číslo zadané jako text.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestCisloText",
 "lastName": "Studium",
 "email": "test.cislo.text.20261029@example.com",
 "numberOfFailedStudies": "3"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity nebo 201 Created
Očekávané tělo odpovědi: {"id": 395,"firstName": "TestCisloText","lastName": "STUDIUM","email": "test.cislo.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 3}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE email = 'test.cislo.text.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_028 – Desetinné číslo zadané jako text

Cíl: Desetinné číslo zadané jako text.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestDesetinneText",
 "lastName": "Studium",
 "email": "test.desetinne.text.20261029@example.com",
 "numberOfFailedStudies": "3.5"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  number_of_failed_studies
FROM public.students
WHERE email = 'test.desetinne.text.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_029 – Příliš vysoký počet neúspěšných studií

Cíl: Příliš vysoký počet neúspěšných studií.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestVelkeCislo",
 "lastName": "Studium",
 "email": "test.velke.cislo.20261029@example.com",
 "numberOfFailedStudies": 2147483648
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  number_of_failed_studies
FROM public.students
WHERE email = 'test.velke.cislo.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_030 – Maximální hodnota 32bitového integeru

Cíl: Maximální hodnota 32bitového integeru.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestMaxInteger",
 "lastName": "Studium",
 "email": "test.max.integer.20261029@example.com",
 "numberOfFailedStudies": 2147483647
}
```

Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 396,"firstName": "TestMaxInteger","lastName": "STUDIUM","email": "test.max.integer.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 2147483647}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE email = 'test.max.integer.20261029@example.com';
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_031 – Velké číslo zadané jako text

Cíl: Velké číslo zadané jako text.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestVelkeCisloText",
 "lastName": "Studium",
 "email": "test.velke.cislo.text.20261029@example.com",
 "numberOfFailedStudies": "2147483648"
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE email = 'test.velke.cislo.text.20261029@example.com';
```

Očekávaný stav databáze: Neúspěšný požadavek nesmí vytvořit studenta s e-mailem test.velke.cislo.text.20261029@example.com; SQL vrátí 0 řádků.

TC_POST_032 – Velmi dlouhé příjmení

Cíl: Velmi dlouhé příjmení.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestDlouhePrijmeni",
 "lastName":
"NNNN
```

TC_POST_034 – Duplicitní e-mail s odlišnou velikostí písmen

Cíl: Duplicitní e-mail s odlišnou velikostí písmen.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestDuplicitaCase",
 "lastName": "Email",
 "email": "TEST.VELKA.PISMENA.20261029@EXAMPLE.COM"
}
```

Očekávaný HTTP stav: 400 Bad Request
Očekávané tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   email
FROM public.students
WHERE LOWER(email) = LOWER('test.velka.pismena.20261029@example.com')
ORDER BY id;
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_035 – E-mail s velkými písmeny a ověřenou duplicitou

Cíl: E-mail s velkými písmeny a ověřenou duplicitou.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestDuplicitaLower",
 "lastName": "Email",
 "email": "test.velka.pismena.20261029@example.com"
}
```

Očekávaný HTTP stav: 400 Bad Request
Očekávané tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   email
FROM public.students
WHERE LOWER(email) = 'test.velka.pismena.20261029@example.com'
ORDER BY id;
```

Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_036 – E-mail s mezerami na začátku a konci

Cíl: E-mail s mezerami na začátku a konci.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestEmailMezery",
 "lastName": "Student",
 "email": "  test.email.mezery.20261029@example.com  "
}
```
Očekávaný HTTP stav: 201 Created při přijetí e-mailu s mezerami; 422 Unprocessable Entity při zapnuté validaci formátu
Očekávané tělo odpovědi: Při 201: JSON nového studenta s e-mailem a ostatními údaji podle zpracované vstupní hodnoty. Při 422: JSON s `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   email,
   LENGTH(email) AS delka_emailu,
   LENGTH(TRIM(email)) AS delka_bez_mezer,
   email = TRIM(email) AS bez_okolnich_mezer
FROM public.students
WHERE TRIM(email) = 'test.email.mezery.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_037 – Duplicitní e-mail s okolními mezerami

Cíl: Duplicitní e-mail s okolními mezerami.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestDuplicitaMezery",
 "lastName": "Student",
 "email": "test.email.mezery.20261029@example.com"
}
```
Očekávaný HTTP stav: 400 Bad Request
Očekávané tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   email,
   LENGTH(email) AS delka_emailu,
   LENGTH(TRIM(email)) AS delka_bez_mezer
FROM public.students
WHERE TRIM(email) = 'test.email.mezery.20261029@example.com'
ORDER BY id;
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_038 – E-mail obsahující pouze mezery

Cíl: E-mail obsahující pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestEmailPouzeMezery",
 "lastName": "Student",
 "email": "   "
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail":"body -> email: Validation error","code":"VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email,
   LENGTH(email) AS delka_emailu,
   LENGTH(TRIM(email)) AS delka_bez_mezer
FROM public.students
WHERE first_name = 'TestEmailPouzeMezery'
ORDER BY id DESC;
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_039 – E-mail s tabulátory

Cíl: E-mail s tabulátory.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestEmailTab",
 "lastName": "Student",
 "email": "\t\t"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail":"body -> email: Validation error","code":"VALIDATION_ERROR"} – očekávané odmítnutí e-mailu složeného pouze z tabulátorů.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   email,
   LENGTH(email) AS delka_emailu,
   ENCODE(CONVERT_TO(email, 'UTF8'), 'hex') AS email_hex
FROM public.students
WHERE first_name = 'TestEmailTab'
ORDER BY id DESC;
```
Očekávaný stav databáze: Při očekávaném odmítnutí 0 nových záznamů; při neočekávaném HTTP 201 zkontrolovat v DB nového studenta a hodnotu email_hex = 0909.

TC_POST_040 – Záporné ID zadané jako text

Cíl: Záporné ID zadané jako text.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "id": "abc",
 "firstName": "TestIdText",
 "lastName": "Student",
 "email": "test.id.text.20261029@example.com"
}
```
Očekávaný HTTP stav: 201 Created nebo 422 Unprocessable Entity
Očekávané tělo odpovědi: {"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE email = 'test.id.text.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_041 – Hodnota `null` v poli `email`

Cíl: Hodnota `null` v poli `email`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNullEmailPost",
 "lastName": "Student",
 "email": null,
 "numberOfFailedStudies": 0
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 404,"firstName": "TestNullEmailPost","lastName": "STUDENT","email": null,"isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  email IS NULL AS email_je_null,
  number_of_failed_studies
FROM public.students
WHERE first_name = 'TestNullEmailPost'
ORDER BY id DESC;
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_042 – Hodnota `null` v poli `numberOfFailedStudies`

Cíl: Hodnota `null` v poli `numberOfFailedStudies`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNullFailedStudies",
 "lastName": "Student",
 "email": "test.null.failed.studies.20261029@example.com",
 "numberOfFailedStudies": null
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 405,"firstName": "TestNullFailedStudies","lastName": "STUDENT","email": "test.null.failed.studies.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  number_of_failed_studies,
  number_of_failed_studies IS NULL AS hodnota_je_null
FROM public.students
WHERE email = 'test.null.failed.studies.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_043 – Hodnota `null` v poli `isEuCitizen`

Cíl: Hodnota `null` v poli `isEuCitizen`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNullEuCitizen",
 "lastName": "Student",
 "email": "test.null.eu.citizen.20261029@example.com",
 "isEuCitizen": null
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 406,"firstName": "TestNullEuCitizen","lastName": "STUDENT","email": "test.null.eu.citizen.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  is_eu_citizen IS NULL AS hodnota_je_null
FROM public.students
WHERE email = 'test.null.eu.citizen.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_044 – Neplatný datový typ `firstName` – pole

Cíl: Neplatný datový typ `firstName` – pole.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": ["Jan", "Petr"],
 "lastName": "TestArray",
 "email": "test.array.firstname.20261029@example.com"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE email = 'test.array.firstname.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_045 – Neplatný datový typ `lastName` – objekt

Cíl: Neplatný datový typ `lastName` – objekt.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestObjectLastName",
 "lastName": {
  "value": "Novak"
 },
 "email": "test.object.lastname.20261029@example.com"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> lastName: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE email = 'test.object.lastname.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_046 – Prázdný řetězec v poli `lastName`

Cíl: Prázdný řetězec v poli `lastName`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestPrazdnePrijmeni",
 "lastName": "",
 "email": "test.prazdne.prijmeni.20261029@example.com"
}
```
Očekávaný HTTP stav: 201 Created při normalizaci prázdného příjmení na null; 422 Unprocessable Entity jen pokud je neprázdné příjmení povinné
Očekávané tělo odpovědi: Při 201: JSON nového studenta, `lastName:null` po normalizaci prázdné hodnoty; při 422: JSON s `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  LENGTH(last_name) AS delka_prijmeni,
  email
FROM public.students
WHERE email = 'test.prazdne.prijmeni.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_047 – Prázdný řetězec v poli `firstName`

Cíl: Prázdný řetězec v poli `firstName`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "",
 "lastName": "TestPrazdneJmeno",
 "email": "test.prazdne.jmeno.20261029@example.com"
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 408,"firstName": "","lastName": "TESTPRAZDNEJMENO","email": "test.prazdne.jmeno.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  first_name IS NULL AS jmeno_je_null,
  LENGTH(first_name) AS delka_jmena,
  last_name,
  email
FROM public.students
WHERE email = 'test.prazdne.jmeno.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_048 – Prázdný řetězec v poli `email`

Cíl: Prázdný řetězec v poli `email`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestPrazdnyEmail",
 "lastName": "Student",
 "email": ""
}
```
Očekávaný HTTP stav: 201 Created
Očekávané tělo odpovědi: {"id": 409,"firstName": "TestPrazdnyEmail","lastName": "STUDENT","email": "","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  email IS NULL AS email_je_null,
  LENGTH(email) AS delka_emailu
FROM public.students
WHERE first_name = 'TestPrazdnyEmail'
ORDER BY id DESC;
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_049 – Neplatný typ `isEuCitizen` – JSON objekt

Cíl: Neplatný typ `isEuCitizen` – JSON objekt.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestBooleanObject",
 "lastName": "Student",
 "email": "test.boolean.object.20261029@example.com",
 "isEuCitizen": {
  "value": true
 }
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen
FROM public.students
WHERE email = 'test.boolean.object.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_050 – Duplicitní e-mail s `null` jménem

Cíl: Duplicitní e-mail s `null` jménem.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": null,
 "lastName": "TestDuplicitaNull",
 "email": "test.id.text.20261029@example.com"
}
```
Očekávaný HTTP stav: 400 Bad Request
Očekávané tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email
FROM public.students
WHERE email = 'test.id.text.20261029@example.com'
ORDER BY id;
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_POST_051 – Neplatný typ `numberOfFailedStudies` – JSON pole

Cíl: Neplatný typ `numberOfFailedStudies` – JSON pole.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/students.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit vytvoření a hodnoty záznamu, nebo že při odmítnutí záznam nevznikl.

Metoda a URL: POST {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestFailedStudiesArray",
 "lastName": "Student",
 "email": "test.failed.studies.array.20261029@example.com",
 "numberOfFailedStudies": [1, 2, 3]
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE email = 'test.failed.studies.array.20261029@example.com';
```
Očekávaný stav databáze: Při 201 vznikne právě požadovaný záznam se správnými hodnotami; při chybě nevznikne nový záznam.

TC_PUT_001 – Úprava existujícího studenta

Cíl: Úprava existujícího studenta.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "JanUpraveny",
  "lastName": "Novak",
  "email": "testjan.novak.20261029@example.com",
  "isEuCitizen": true,
  "zipCode": "27201",
  "numberOfFailedStudies": 1
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 376,"firstName": "JanUpraveny","lastName": "Novak","email": "testjan.novak.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 1}
SQL dotaz v DBeaveru:

```
SELECT *
FROM public.students
WHERE id = 376;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_002 – Částečná aktualizace studenta

Cíl: Částečná aktualizace studenta.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "numberOfFailedStudies": 2
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 376,"firstName": "JanUpraveny","lastName": "Novak","email": "testjan.novak.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 2}
SQL dotaz v DBeaveru:

```
SELECT *
FROM public.students
WHERE id = 376;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_003 – Záporný počet neúspěšných studií

Cíl: Záporný počet neúspěšných studií.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "numberOfFailedStudies": -1
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, number_of_failed_studies
FROM public.students
WHERE id = 376;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_004 – Aktualizace neexistujícího studenta

Cíl: Aktualizace neexistujícího studenta.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/999999999
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "NeexistujiciStudent"
}
```
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze zmenit data neexistujuceho studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru:
```
SELECT *
FROM public.students
WHERE id = 999999999;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_005 – Neplatný datový typ

Cíl: Neplatný datový typ.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "numberOfFailedStudies": "abc"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, number_of_failed_studies
FROM public.students
WHERE id = 376;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_006 – Překročení maximální délky jména

Cíl: Překročení maximální délky jména.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
{
"firstName":
"AAAA

TC_PUT_008 – Aktualizace hodnotou NULL

Cíl: Aktualizace hodnotou NULL.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": null
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 376` a `numberOfFailedStudies: null`; ostatní pole mají zůstat beze změny oproti stavu před PUT.
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  number_of_failed_studies
FROM public.students
WHERE id = 376;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_009 – Prázdné JSON tělo

Cíl: Prázdné JSON tělo.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/376.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/376
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: {}
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 376;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_010 – Duplicitní e-mail při aktualizaci

Cíl: Duplicitní e-mail při aktualizaci.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/377.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "email": "testjan.novak.20261029@example.com"
}
```
Očekávaný HTTP stav: 400 Bad Request nebo 409 Conflict
Očekávané tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru:
```
SELECT id, first_name, email
FROM public.students
WHERE id IN (376, 377)
ORDER BY id;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_011 – Pokus o změnu ID studenta

Cíl: Pokus o změnu ID studenta.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/377.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "id": 999999,
 "firstName": "TestZmenaID"
}
```
Očekávaný HTTP stav: 200 OK při ignorování pokusu o změnu id; 422 Unprocessable Entity při odmítnutí nepovoleného pole
Očekávané tělo odpovědi: Při 200: JSON studenta s původním `id` a upraveným `firstName`. Při 422: JSON s `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru:
```
SELECT id, first_name, email
FROM public.students
WHERE id IN (377, 999999)
ORDER BY id;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_012 – Aktualizace prázdného řetězce

Cíl: Aktualizace prázdného řetězce.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/377.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": ""
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> firstName: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 377;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_013 – Změna neplatného e-mailu

Cíl: Změna neplatného e-mailu.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/377.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "email": "neplatny-email-put"
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 377` a `email: "neplatny-email-put"`; ostatní pole mají zůstat beze změny.
SQL dotaz v DBeaveru:
```
SELECT id, first_name, email
FROM public.students
WHERE id = 377;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_014 – Nechtěná změna `isEuCitizen`

Cíl: Nechtěná změna `isEuCitizen`.
Postup testu:
1. Nejprve PUT /students/377 s {"isEuCitizen":true}; ověřit true v DB. Poté PUT /students/377 pouze s {"firstName":"TestAktualizace"}.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestAktualizace"
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id":377,"firstName":"TestAktualizace","isEuCitizen":true} (ostatní údaje zachovány)
SQL dotaz v DBeaveru:
```
SELECT id, first_name, email, is_eu_citizen
FROM public.students
WHERE id = 377;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_015 – Přepis dalších neuvedených polí

Cíl: Přepis dalších neuvedených polí.
Postup testu:
1. Nejprve PUT /students/377 s {"isEuCitizen":true,"numberOfFailedStudies":3}. Poté PUT pouze s {"firstName":"TestZachovaniHodnot"}.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestZachovaniHodnot"
}
```
Očekávaný HTTP stav: 200 OK – změnit pouze firstName, isEuCitizen a numberOfFailedStudies zachovat
Očekávané tělo odpovědi: {"id":377,"firstName":"TestZachovaniHodnot","isEuCitizen":true,"numberOfFailedStudies":3} (ostatní pole zachována)
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   is_eu_citizen,
   number_of_failed_studies
FROM public.students
WHERE id = 377;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_016 – Aktualizace pouze počtu neúspěšných studií

Cíl: Aktualizace pouze počtu neúspěšných studií.
Postup testu:
1. Nejprve PUT /students/377 s {"isEuCitizen":true} a ověřit true / 3 v DB. Poté PUT pouze s {"numberOfFailedStudies":4}.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": 4
}
```
Očekávaný HTTP stav: 200 OK – změnit pouze numberOfFailedStudies, isEuCitizen zachovat
Očekávané tělo odpovědi: {"id":377,"isEuCitizen":true,"numberOfFailedStudies":4} (ostatní pole zachována)
SQL dotaz v DBeaveru:
```
SELECT id, is_eu_citizen, number_of_failed_studies
FROM public.students
WHERE id = 377;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_017 – Zachování PSČ při aktualizaci

Cíl: Zachování PSČ při aktualizaci.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/377.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/377
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "zipCode": "27201"
}
```
Očekávaný HTTP stav: 200 OK – bez změny ostatních polí; zipCode uložit, pokud ho API podporuje
Očekávané tělo odpovědi: Při 200: JSON studenta ID 377; ostatní hodnoty zachovány a podporované `zipCode` uloženo jako `27201`.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   zip_code,
   is_eu_citizen,
   number_of_failed_studies
FROM public.students
WHERE id = 377;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_018 – Zachování příjmení při změně jména

Cíl: Zachování příjmení při změně jména.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestZmenyJmena"
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382`, `firstName: "TestZmenyJmena"`, `lastName: "STUDENT"` a původním `isEuCitizen: true`.
SQL dotaz v DBeaveru:
```
SELECT
    id,
    first_name,
    last_name,
    email,
    is_eu_citizen,
    number_of_failed_studies
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_019 – Nastavení příjmení na NULL

Cíl: Nastavení příjmení na NULL.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "lastName": null
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: Při povoleném NULL JSON objekt s `id: 382` a `lastName: null`; všechna ostatní pole beze změny.
SQL dotaz v DBeaveru:
```
SELECT
    id,
    first_name,
    last_name,
    is_eu_citizen
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_020 – PUT s hodnotou null v poli lastName

Cíl: PUT s hodnotou null v poli lastName.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNullPrijmeni",
 "lastName": null
}
```
Očekávaný HTTP stav: 200 OK při přijetí null v lastName; 422 Unprocessable Entity při zákazu null
Očekávané tělo odpovědi: Při 200: JSON studenta s `lastName:null` a zachovanými ostatními poli; při 422: JSON s validační chybou.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   is_eu_citizen
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_021 – Prázdný e-mail

Cíl: Prázdný e-mail.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "email": ""
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> email: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   email,
   is_eu_citizen
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_022 – Příliš dlouhý e-mail

Cíl: Příliš dlouhý e-mail.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "email":
"aaaa
```

TC_PUT_024 – Neplatný datový typ e-mailu

Cíl: Neplatný datový typ e-mailu.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "email": 12345
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> email: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   LENGTH(email) AS delka_emailu,
   email,
   is_eu_citizen
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_025 – E-mail nastavený na `null`

Cíl: E-mail nastavený na `null`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNullEmail",
 "email": null
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: Při povoleném NULL JSON objekt s `id: 382`, `firstName: "TestNullEmail"` a `email: null`; ostatní hodnoty se nemají změnit.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   email,
   LENGTH(email) AS delka_emailu,
   email IS NULL AS email_je_null,
   is_eu_citizen
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_026 – Neplatná hodnota `isEuCitizen`

Cíl: Neplatná hodnota `isEuCitizen`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "isEuCitizen": 2
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean, unable to interpret input","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   LENGTH(email) AS delka_emailu,
   is_eu_citizen
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_027 – Hodnota `null` u isEuCitizen

Cíl: Hodnota `null` u isEuCitizen.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNullBoolean",
 "isEuCitizen": null
}
```
Očekávaný HTTP stav: 200 OK při ignorování hodnoty null bez nechtěné změny boolean pole; případně 422 Unprocessable Entity při odmítnutí null.
Očekávané tělo odpovědi: Při 200: JSON s firstName="TestNullBoolean" a isEuCitizen=false (nebo null, umožňuje-li to API); při 422: validační chyba. Hodnota true není očekávaná.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   is_eu_citizen,
   is_eu_citizen IS NULL AS eu_je_null,
   LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 382;
```
Očekávaný stav databáze: Před PUT je is_eu_citizen=false. Přijetí null nesmí svévolně změnit boolean na true; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_028 – Aktualizace na stejnou hodnotu

Cíl: Aktualizace na stejnou hodnotu.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "TestNullBoolean"
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382`, `firstName: "TestNullBoolean"` a nezměněnou původní hodnotou `isEuCitizen: true`.
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    last_name,
    is_eu_citizen,
    LENGTH(email) AS delka_emailu,
    number_of_failed_studies
FROM public.students
WHERE id = 382;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_029 – Opakovaný identický požadavek

Cíl: Opakovaný identický požadavek.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "TestNullBoolean"
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382`, `firstName: "TestNullBoolean"` a stále `isEuCitizen: false` (opakovaný identický PUT nesmí hodnotu přepnout).
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    is_eu_citizen,
    LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 382;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_030 – Explicitní nastavení `isEuCitizen`

Cíl: Explicitní nastavení `isEuCitizen`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "isEuCitizen": true
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382` a `isEuCitizen: true`, jak bylo výslovně uvedeno v požadavku; ostatní pole beze změny.
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    is_eu_citizen,
    LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 382;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_031 – Explicitní nastavení `false`

Cíl: Explicitní nastavení `false`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "isEuCitizen": false
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382` a `isEuCitizen: false`, jak bylo výslovně uvedeno v požadavku; ostatní pole beze změny.
SQL dotaz v DBeaveru:

```
SELECT
    id,
    first_name,
    is_eu_citizen,
    LENGTH(email) AS delka_emailu,
    number_of_failed_studies
FROM public.students
WHERE id = 382;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_032 – Ověření přepínání hodnoty

Cíl: Ověření přepínání hodnoty.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "isEuCitizen": false
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 382,"firstName": "TestNullBoolean","lastName": "STUDENT","email": "aaaa

TC_PUT_034 – Aktualizace více polí současně

Cíl: Aktualizace více polí současně.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestVicePoli",
 "lastName": "Aktualizovany",
 "numberOfFailedStudies": 5,
 "isEuCitizen": true
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382`, `firstName: "TestVicePoli"`, `lastName: "Aktualizovany"`, `isEuCitizen: true`, `numberOfFailedStudies: 5`; e-mail zůstane původní.
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   number_of_failed_studies,
   is_eu_citizen,
   LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 382;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_035 – Opakování aktualizace více polí

Cíl: Opakování aktualizace více polí.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/382.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/382
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestVicePoli",
 "lastName": "Aktualizovany",
 "numberOfFailedStudies": 5,
 "isEuCitizen": true
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 382`, `firstName: "TestVicePoli"`, `lastName: "Aktualizovany"`, `isEuCitizen: true`, `numberOfFailedStudies: 5`; opakovaný PUT zachová stejný stav.
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   number_of_failed_studies,
   is_eu_citizen,
   LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 382;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_036 – Převod příjmení při aktualizaci

Cíl: Převod příjmení při aktualizaci.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/384.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/384
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "lastName": "Svobodova"
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: JSON objekt studenta s `id: 384`, aktualizovaným `lastName: "Svobodova"` a zachovaným `isEuCitizen: false`; ostatní údaje beze změny.
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 384;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_037 – Aktualizace studenta s neplatným ID

Cíl: Aktualizace studenta s neplatným ID.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/999999999
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNeexistujiciStudent"
}
```
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze zmenit data neexistujuceho studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE id = 999999999;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_038 – Aktualizace s neplatným desetinným ID

Cíl: Aktualizace s neplatným desetinným ID.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.5.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403.5
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "TestNeplatneDesetinneID"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_039 – Aktualizace studenta s příliš dlouhým jménem

Cíl: Aktualizace studenta s příliš dlouhým jménem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName":
"AAAA
```

TC_PUT_040 – Aktualizace příjmení na 256 znaků

Cíl: Aktualizace příjmení na 256 znaků.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "lastName":
"NNNN
```

TC_PUT_042 – Překročení maximální hodnoty celého čísla

Cíl: Překročení maximální hodnoty celého čísla.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": 2147483648
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_043 – Neplatný datový typ počtu studií

Cíl: Neplatný datový typ počtu studií.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": "abc"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_044 – Desetinné číslo v počtu neúspěšných studií

Cíl: Desetinné číslo v počtu neúspěšných studií.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": 2.5
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be a valid integer, got a number with a fractional part","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_045 – Neplatný datový typ `isEuCitizen`

Cíl: Neplatný datový typ `isEuCitizen`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "isEuCitizen": 2
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean, unable to interpret input","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_046 – Aktualizace s prázdným JSON objektem

Cíl: Aktualizace s prázdným JSON objektem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: {}
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Value error, At least one field must be provided for update","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_047 – Aktualizace s neplatným JSON

Cíl: Aktualizace s neplatným JSON.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": "Test
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> 23: JSON decode error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_048 – Aktualizace s JSON polem místo objektu

Cíl: Aktualizace s JSON polem místo objektu.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
[
  {"firstName": "TestPole"}
]
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Input should be a valid dictionary or object to extract fields from","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_049 – Aktualizace s duplicitním e-mailem

Cíl: Aktualizace s duplicitním e-mailem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
  {
    "email": "TEST.VELKA.PISMENA.20261029@EXAMPLE.COM"
  }
```
Očekávaný HTTP stav: 400 Bad Request nebo 409 Conflict; duplicitní e-mail musí být odmítnut.
Očekávané tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
SQL dotaz v DBeaveru:
```
  SELECT id, first_name, email
  FROM public.students
  WHERE id IN (398, 403)
  ORDER BY id;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_050 – E-mail s neplatným formátem

Cíl: E-mail s neplatným formátem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
  {
    "email": "neplatny-email-bez-zavinace"
  }
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> email: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
  SELECT id, first_name, email
  FROM public.students
  WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_051 – E-mail obsahující pouze mezery

Cíl: E-mail obsahující pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "email": "   "
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> email: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   email,
   LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_052 – Aktualizace e-mailu na hodnotu `null`

Cíl: Aktualizace e-mailu na hodnotu `null`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "email": null
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Value error, At least one field must be provided for update","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   email,
   LENGTH(email) AS delka_emailu
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_053 – Aktualizace s hodnotou `true` místo čísla

Cíl: Aktualizace s hodnotou `true` místo čísla.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": true
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_054 – Aktualizace čísla pomocí textové hodnoty

Cíl: Aktualizace čísla pomocí textové hodnoty.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": "3"
}
```
Očekávaný HTTP stav: 200 OK s převodem řetězce "3" na číslo 3; 422 Unprocessable Entity při striktní validaci typu
Očekávané tělo odpovědi: Při 200: JSON studenta ID 403 s `numberOfFailedStudies:3`; při 422: `{ "code":"VALIDATION_ERROR" }`.
SQL dotaz v DBeaveru:
```
SELECT
  id,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_055 – Překročení rozsahu čísla zadaného jako text

Cíl: Překročení rozsahu čísla zadaného jako text.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "numberOfFailedStudies": "2147483648"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Validation error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_056 – Aktualizace s nulovou hodnotou

Cíl: Aktualizace s nulovou hodnotou.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "numberOfFailedStudies": 0
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT id, number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_057 – Aktualizace s ID obsahujícím počáteční nuly

Cíl: Aktualizace s ID obsahujícím počáteční nuly.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/000403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/000403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestPocatecniNuly"
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "TestPocatecniNuly","lastName": "STUDENT","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_058 – Aktualizace s desetinným zápisem celého ID

Cíl: Aktualizace s desetinným zápisem celého ID.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.0.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403.0
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestDesetinneID"
}
```
Očekávaný HTTP stav: 200 OK, pokud API přijme normalizovaný číselný identifikátor; 422 Unprocessable Entity při striktní validaci path
Očekávané tělo odpovědi: Při 200 JSON objekt aktualizovaného studenta s `id: 403` a hodnotami upravených polí; při 422 JSON s `code: "VALIDATION_ERROR"` pro neplatný formát ID v URL.
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_059 – Aktualizace s ID mimo rozsah 32bitového integeru

Cíl: Aktualizace s ID mimo rozsah 32bitového integeru.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/2147483648.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/2147483648
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestVelkeID"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity při kontrole rozsahu ID, případně 404 Not Found; nikoli 500 Internal Server Error.
Očekávané tělo odpovědi: Při 422 JSON s kódem VALIDATION_ERROR pro parametr student_id; při 404 JSON s kódem NOT_FOUND. Nesmí vracet INTERNAL_SERVER_ERROR.
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE first_name = 'TestVelkeID';
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_060 – Aktualizace s neplatným textovým ID

Cíl: Aktualizace s neplatným textovým ID.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/abc.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/abc
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestNeplatneID"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_061 – Aktualizace s negativním ID

Cíl: Aktualizace s negativním ID.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/-1.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/-1
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestZaporneID"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity nebo 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze zmenit data neexistujuceho studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = -1;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_062 – Aktualizace s ID obsahujícím mezeru

Cíl: Aktualizace s ID obsahujícím mezeru.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/%20%20%20.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/%20%20%20
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestMezeryID"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_063 – Aktualizace s ID obsahujícím SQL výraz

Cíl: Aktualizace s ID obsahujícím SQL výraz.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403%20OR%201%3D1.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403%20OR%201%3D1
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestSQLInjection"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_064 – Aktualizace s neznámým polem

Cíl: Aktualizace s neznámým polem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": "TestNeznamePole",
  "favoriteColor": "blue"
}
```
Očekávaný HTTP stav: 200 OK při ignorování neznámého pole favoriteColor; 422 Unprocessable Entity při zákazu extra polí
Očekávané tělo odpovědi: Při 200 JSON studenta s `id: 403` a s aktualizovanými známými poli, bez ukládání neznámého `favoriteColor`; při 422 JSON s `code: "VALIDATION_ERROR"` pro nepovolené extra pole.
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   last_name,
   email
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_065 – Aktualizace s pokusem změnit ID studenta

Cíl: Aktualizace s pokusem změnit ID studenta.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "id": 999999,
 "firstName": "TestZmenaID"
}
```
Očekávaný HTTP stav: 200 OK při ignorování id v těle, případně 422 Unprocessable Entity při odmítnutí tohoto pole
Očekávané tělo odpovědi: Při HTTP 200: JSON aktualizovaného studenta s id=403 a firstName="TestZmenaID"; pole id=999999 se nesmí převzít. Při HTTP 422: JSON validační chyby (VALIDATION_ERROR).
SQL dotaz v DBeaveru:
```
SELECT id, first_name, last_name, email
FROM public.students
WHERE id IN (403, 999999)
ORDER BY id;
```
Očekávaný stav databáze: Při HTTP 200 zůstává původní ID 403 a aktualizuje se pouze first_name; nový student 999999 nesmí vzniknout. Při HTTP 422 se záznam nemění.

TC_PUT_066 – Aktualizace s prázdným jménem

Cíl: Aktualizace s prázdným jménem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": ""
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "","lastName": "STUDENT","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
   id,
   first_name,
   LENGTH(first_name) AS delka_jmena,
   first_name IS NULL AS je_null
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_067 – Aktualizace s prázdným příjmením

Cíl: Aktualizace s prázdným příjmením.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "lastName": ""
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "","lastName": "","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  last_name,
  LENGTH(last_name) AS delka_prijmeni,
  last_name IS NULL AS je_null
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_068 – Aktualizace jména obsahujícího pouze mezery

Cíl: Aktualizace jména obsahujícího pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "  "
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": " ","lastName": "","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  LENGTH(first_name) AS delka_jmena,
  first_name IS NULL AS je_null
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_069 – Aktualizace jména s českou diakritikou

Cíl: Aktualizace jména s českou diakritikou.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": "Jiří",
 "lastName": "Dvořáková"
}
```
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "Jiří","lastName": "Dvořáková","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_070 – Aktualizace jména pomocí hodnoty `false`

Cíl: Aktualizace jména pomocí hodnoty `false`.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "firstName": false
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_071 – Aktualizace příjmení pomocí pole

Cíl: Aktualizace příjmení pomocí pole.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
 "lastName": ["Novák", "Svoboda"]
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> lastName: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT id, last_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_072 – Aktualizace s neplatným datovým typem e-mailu

Cíl: Aktualizace s neplatným datovým typem e-mailu.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "email": {
  "address": "test@example.com"
 }
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> email: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT id, email
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_073 – Aktualizace s hodnotou `isEuCitizen` jako pole

Cíl: Aktualizace s hodnotou `isEuCitizen` jako pole.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "isEuCitizen": [true, false]
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_074 – Aktualizace s prázdným textovým tělem

Cíl: Aktualizace s prázdným textovým tělem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
Tělo prázdné: Body → raw → JSON, žádné znaky (nikoli `{}`).
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_075 – Aktualizace bez autentizačního tokenu

Cíl: Aktualizace bez autentizačního tokenu.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. Nastavit autorizaci (Bez Authorization hlavičky (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bez Authorization hlavičky (No Auth).
Testovací data / JSON:
```
{
  "firstName": "TestBezAutorizace"
}
```
Očekávaný HTTP stav: 401 Unauthorized nebo 403 Forbidden – nepovolený přístup bez tokenu
Očekávané tělo odpovědi: Při 403 {"detail":"Not authenticated","code":"FORBIDDEN"}; při 401 odpovídající autorizační chyba. V obou případech nesmí dojít k zápisu ani smazání studenta.
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_076 – Aktualizace s neplatným tokenem

Cíl: Aktualizace s neplatným tokenem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** neplatny_token_12345 – záměrně neplatný token) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: neplatny_token_12345 (záměrně neplatný token).
Testovací data / JSON:
```
{
  "firstName": "TestNeplatnyToken"
}
```
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:
```
SELECT id, first_name
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_077 – Aktualizace s prošlým tokenem

Cíl: Aktualizace s prošlým tokenem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: skutečný dříve platný, nyní expirovaný JWT; nepoužívat platný {{token}} ani náhodný řetězec.
Testovací data / JSON:
```
{
  "firstName": "TestExpirovanyToken"
}
```
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_078 – Aktualizace s hodnotou `null` v JSON těle

Cíl: Aktualizace s hodnotou `null` v JSON těle.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
null
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_079 – Aktualizace s více neplatnými poli současně

Cíl: Aktualizace s více neplatnými poli současně.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
```
{
  "firstName": 123,
  "lastName": ["Novák"],
  "email": true,
  "numberOfFailedStudies": -5,
  "isEuCitizen": "neplatna_hodnota"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string; body -> lastName: Input should be a valid string; body -> email: Input should be a valid string; body -> isEuCitizen: Input should be a valid boolean, unable to interpret input; body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:
```
SELECT
  id,
  first_name,
  last_name,
  email,
  is_eu_citizen,
  number_of_failed_studies
FROM public.students
WHERE id = 403;
```
Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_080 – Aktualizace s neplatným JSON a chybějícím tokenem

Cíl: Aktualizace s neplatným JSON a chybějícím tokenem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (No Auth**); odstranit hlavičku Authorization, nastavit Content-Type: application/json a do Body → raw vložit níže uvedená záměrně neplatná testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: No Auth; hlavička Authorization se neposílá, tělo JSON je současně záměrně neplatné.
Testovací data / JSON:

```
{
 "firstName": "Test
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity pro neplatný JSON; 401 Unauthorized/403 Forbidden, pokud autentizace předchází validaci
Očekávané tělo odpovědi: {"code":"VALIDATION_ERROR","detail":"chyba parsování neplatného JSON"} (popis v detail se může lišit)
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name, email
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_081 – Neplatný token a platné aktualizační údaje

Cíl: Neplatný token a platné aktualizační údaje.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** neplatný testovací token.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: neplatný testovací token.
Testovací data / JSON:

```
{
 "firstName": "TestNeplatnyTokenValidniData"
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_082 – Aktualizace s neplatným tokenem a neexistujícím studentem

Cíl: Aktualizace s neplatným tokenem a neexistujícím studentem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** neplatny_token_12345 – záměrně neplatný token) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/999999999
Autorizace: Bearer Token: neplatny_token_12345 (záměrně neplatný token).
Testovací data / JSON:

```
{
  "firstName": "TestNeexistujiciBezOpravneni"
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name
FROM public.students
WHERE id = 999999999
   OR first_name = 'TestNeexistujiciBezOpravneni';
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_083 – Neexistující student a neplatná data

Cíl: Neexistující student a neplatná data.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/999999999
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName": 123,
  "numberOfFailedStudies": -5
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity – neplatné hodnoty musí odmítnout celý požadavek
Očekávané tělo odpovědi: {"code":"VALIDATION_ERROR","detail":"neplatný typ firstName a záporné numberOfFailedStudies"} (popis v detail se může lišit)
SQL dotaz v DBeaveru:

```
SELECT id, first_name, number_of_failed_studies
FROM public.students
WHERE id = 999999999;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_084 – Aktualizace s duplicitním polem v JSON

Cíl: Aktualizace s duplicitním polem v JSON.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "PrvniHodnota",
 "firstName": "DruhaHodnota"
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "DruhaHodnota","lastName": "Dvořáková","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_085 – Aktualizace s hodnotou `isEuCitizen` jako text

Cíl: Aktualizace s hodnotou `isEuCitizen` jako text.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "isEuCitizen": "true"
}
```

Očekávaný HTTP stav: 200 OK s převodem řetězce "true" na boolean true; 422 Unprocessable Entity při striktní validaci typu
Očekávané tělo odpovědi: Při 200: JSON studenta ID 403 s `isEuCitizen:true`; při 422: `{ "code":"VALIDATION_ERROR" }`.
SQL dotaz v DBeaveru:

```
SELECT
  id,
  is_eu_citizen
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_086 – Aktualizace e-mailu na maximální délku 255 znaků

Cíl: Aktualizace e-mailu na maximální délku 255 znaků.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
{
"email":
"aaaa

TC_PUT_088 – Aktualizace jména na maximální délku 255 znaků

Cíl: Aktualizace jména na maximální délku 255 znaků.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
  "firstName":
"AAAA
```

TC_PUT_090 – Aktualizace více polí, z nichž jedno je neplatné

Cíl: Aktualizace více polí, z nichž jedno je neplatné.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "TestCastecnaAktualizace",
 "numberOfFailedStudies": -1
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_091 – Aktualizace více platných polí současně

Cíl: Aktualizace více platných polí současně.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "Petr",
 "lastName": "Novák",
 "email": "petr.novak.test092@example.com",
 "numberOfFailedStudies": 2
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "Petr","lastName": "Novák","email": "petr.novak.test092@example.com","isEuCitizen": false,"numberOfFailedStudies": 2}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_092 – Aktualizace se současně platným a neplatným e-mailem

Cíl: Aktualizace se současně platným a neplatným e-mailem.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:
{
"firstName": "TestNeplatnyEmail",
"email":
"aaaa

TC_PUT_094 – Aktualizace pomocí JSON s prázdným řetězcem a platnou hodnotou

Cíl: Aktualizace pomocí JSON s prázdným řetězcem a platnou hodnotou.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "",
 "lastName": "TestPrazdneJmeno"
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "","lastName": "TestPrazdneJmeno","email": "petr.novak.test092@example.com","isEuCitizen": false,"numberOfFailedStudies": 2}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   LENGTH(first_name) AS delka_jmena,
   last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_PUT_095 – Aktualizace se stejnými hodnotami

Cíl: Aktualizace se stejnými hodnotami.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/students/403.
2. Nastavit autorizaci (Bearer Token: {{token}}.) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit změněné hodnoty a zachování ostatních údajů studenta.

Metoda a URL: PUT {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON:

```
{
 "firstName": "",
 "lastName": "TestPrazdneJmeno",
 "email": "petr.novak.test092@example.com",
 "numberOfFailedStudies": 2
}
```

Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"id": 403,"firstName": "","lastName": "TestPrazdneJmeno","email": "petr.novak.test092@example.com","isEuCitizen": true,"numberOfFailedStudies": 2}
SQL dotaz v DBeaveru:

```
SELECT
   id,
   first_name,
   last_name,
   email,
   number_of_failed_studies
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěchu se změní jen požadovaná pole; při odmítnutí požadavku zůstane záznam beze změny.

TC_DELETE_001 – Smazání existujícího studenta

Cíl: Smazání existujícího studenta.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/379.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/379
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné, nastav none.
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"message": "Student deleted successfully"}
SQL dotaz v DBeaveru:

```
SELECT *
FROM public.students
WHERE id = 379;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_002 – Kontrola dostupnosti studenta po DELETE

Cíl: Ověřit, že po úspěšném DELETE není student dále dostupný přes GET ani uložený v databázi.
Postup testu:
1. V Postmanu odeslat DELETE {{base_url}}/students/379; bezprostředně poté GET {{base_url}}/students/379.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/379; následně GET {{base_url}}/students/379
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné.
Očekávaný HTTP stav: DELETE 200 OK, při následném GET 404 Not Found.
Očekávané tělo odpovědi: DELETE: {"message":"Student deleted successfully"}; následný GET: {"detail":"Chyba, nelze najit studenta","code":"NOT_FOUND"}.
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 379;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_003 – Opakované smazání

Cíl: Opakované smazání.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/379.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/379
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: none
Očekávaný HTTP stav: 404 Not Found při opakovaném DELETE již smazaného studenta (alternativně 204 No Content u idempotentního rozhraní)
Očekávané tělo odpovědi: {"detail":"Chyba, nelze najit studenta","code":"NOT_FOUND"} (již smazaný student)
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 379;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_004 – Smazání neexistujícího studenta

Cíl: Smazání neexistujícího studenta.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/999999999
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: none
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 999999999;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_005 – Neplatný formát ID

Cíl: Neplatný formát ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/abc.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/abc
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: none
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_006 – Smazání bez autentizace

Cíl: Smazání bez autentizace.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/379.
2. Nastavit autorizaci (Bez Authorization hlavičky (No Auth).); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/379
Autorizace: Bez Authorization hlavičky (No Auth).
Testovací data / JSON: Body: none
Očekávaný HTTP stav: 403 Forbidden
Očekávané tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 379;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_007 – Smazání s neplatným tokenem

Cíl: Smazání s neplatným tokenem.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/377.
2. **Nastavit autorizaci (Bearer Token:** neplatny_token_123 – záměrně neplatný token); použít ID uvedené v URL. Tělo požadavku ponechat prázdné.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/377
Autorizace: Bearer Token: neplatny_token_123 (záměrně neplatný token).
Testovací data / JSON: Žádné tělo požadavku (metoda bez JSON body).
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, email
FROM public.students
WHERE id = 377;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_008 – Smazání existujícího studenta a ověření v databázi

Cíl: Smazání existujícího studenta a ověření v databázi.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"message": "Student deleted successfully"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_009 – Smazání studenta s desetinným ID

Cíl: Smazání studenta s desetinným ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403.5.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403.5
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_010 – Překročení maximální hodnoty ID

Cíl: Překročení maximální hodnoty ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/2147483648.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/2147483648
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 2147483648;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_011 – Smazání studenta se záporným ID

Cíl: Smazání studenta se záporným ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/-1.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/-1
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = -1;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_012 – Smazání studenta s ID obsahujícím počáteční nuly

Cíl: Smazání studenta s ID obsahujícím počáteční nuly.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/000403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/000403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"message": "Student deleted successfully"}
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name,
  email
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_013 – Smazání studenta s neplatným tokenem a neexistujícím ID

Cíl: Smazání studenta s neplatným tokenem a neexistujícím ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** neplatny_token_12345 – záměrně neplatný token); použít neexistující ID 999999999 uvedené v URL. Tělo požadavku ponechat prázdné.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/999999999
Autorizace: Bearer Token: neplatny_token_12345 (záměrně neplatný token).
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 999999999;
Očekávaný stav databáze: Žádné smazání; testovaný student 999999999 neexistuje a požadavek s neplatným tokenem nesmí měnit databázi.

TC_DELETE_014 – Neplatný token a existující student

Cíl: Neplatný token a existující student.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** neplatný testovací token.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403
Autorizace: Bearer Token: neplatný testovací token.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Student ID 403 musí po odmítnutí neplatného tokenu zůstat v databázi beze změny.

TC_DELETE_015 – Chybějící token a existující student

Cíl: Chybějící token a existující student.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403.
2. Nastavit autorizaci (Bez Authorization hlavičky (No Auth).); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403
Autorizace: Bez Authorization hlavičky (No Auth).
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized nebo 403 Forbidden – DELETE bez tokenu nesmí projít
Očekávané tělo odpovědi: Při 403 {"detail":"Not authenticated","code":"FORBIDDEN"}; při 401 odpovídající autorizační chyba. V obou případech nesmí dojít k zápisu ani smazání studenta.
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_016 – Chybějící token a neexistující student

Cíl: Chybějící token a neexistující student.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/999999999.
2. Nastavit autorizaci (Bez Authorization hlavičky (No Auth).); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/999999999
Autorizace: Bez Authorization hlavičky (No Auth).
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 403 Forbidden
Očekávané tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 999999999;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_017 – Neplatný token a neexistující student s jiným ID

Cíl: Neplatný token a neexistující student s jiným ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/888888888.
2. **Nastavit autorizaci (Bearer Token:** neplatný testovací token.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/888888888
Autorizace: Bearer Token: neplatný testovací token.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 888888888;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_018 – Opakování TC_DELETE_013

Cíl: Opakování TC_DELETE_013.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/999999999.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/999999999
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 999999999;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_019 – Smazání studenta s ID ve formátu `403.0`

Cíl: Smazání studenta s ID ve formátu `403.0`.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403.0.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403.0
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_020 – SQL injection v ID studenta

Cíl: SQL injection v ID studenta.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403%20OR%201%3D1.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403%20OR%201%3D1
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_021 – Smazání s prázdným ID

Cíl: Smazání s prázdným ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 405 Method Not Allowed – nepodporovaná HTTP metoda
Očekávané tělo odpovědi: {"detail":"Method Not Allowed","code":"UNEXPECTED_ERROR"} (odpověď nepodporované metody DELETE bez ID).
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_022 – Smazání studenta s ID obsahujícím písmena

Cíl: Smazání studenta s ID obsahujícím písmena.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403abc.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403abc
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_023 – Smazání s ID obsahujícím mezery

Cíl: Smazání s ID obsahujícím mezery.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/%20%20%20.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/%20%20%20
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_024 – Smazání studenta s ID začínajícím znaménkem plus

Cíl: Smazání studenta s ID začínajícím znaménkem plus.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/%2B403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/%2B403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK
Očekávané tělo odpovědi: {"message": "Student deleted successfully"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_025 – Neplatný token a neplatné ID

Cíl: Neplatný token a neplatné ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/abc403.
2. **Nastavit autorizaci (Bearer Token:** neplatný testovací token.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/abc403
Autorizace: Bearer Token: neplatný testovací token.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_026 – Platný token a ID s nulou

Cíl: Platný token a ID s nulou.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/0.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/0
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 0;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_027 – Smazání studenta s ID obsahujícím desetinnou čárku

Cíl: Smazání studenta s ID obsahujícím desetinnou čárku.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403%2C0.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403%2C0
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_028 – Smazání studenta s ID ve vědeckém zápisu

Cíl: Smazání studenta s ID ve vědeckém zápisu.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/4.03e2.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/4.03e2
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru:

```
SELECT id, first_name, last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_029 – Neexistující student s maximálním 32bitovým ID

Cíl: Neexistující student s maximálním 32bitovým ID.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/2147483647.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/2147483647
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 2147483647;
Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_030 – Opakované smazání stejného studenta

Cíl: Opakované smazání stejného studenta.
Postup testu:
1. V Postmanu připravit požadavek DELETE {{base_url}}/students/403.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. V DBeaveru spustit uvedený SQL dotaz; ověřit odstranění záznamu, případně nezměněný stav při chybě.

Metoda a URL: DELETE {{base_url}}/students/403
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 200 OK nebo 204 No Content při úspěšném smazání existujícího studenta.
Očekávané tělo odpovědi: Při 200 potvrzení odstranění (např. {"message":"Student deleted successfully"}); při 204 prázdné tělo. SQL SELECT musí vrátit 0 řádků.
SQL dotaz v DBeaveru:

```
SELECT
  id,
  first_name,
  last_name
FROM public.students
WHERE id = 403;
```

Očekávaný stav databáze: Při úspěšném smazání dotaz SELECT nevrátí původního studenta; při chybě záznam zůstává.

TC_DELETE_031 – Ověření existence studenta po smazání pomocí GET

Cíl: Ověření existence studenta po smazání pomocí GET.
Postup testu:
1. Po provedení TC_DELETE_030 připravit GET {{base_url}}/students/403 pro ověření, zda je smazaný student stále dostupný.
2. **Nastavit autorizaci (Bearer Token:** {{token}}.); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat GET a ověřit, zda vrátí 404 Not Found namísto údajů údajně smazaného studenta.
4. Případně potvrdit neexistenci studenta SQL dotazem uvedeným níže; hlavním důkazem tohoto testu je GET po DELETE.

Metoda a URL: GET {{base_url}}/students/403 (po předchozím DELETE v TC_DELETE_030)
Autorizace: Bearer Token: {{token}}.
Testovací data / JSON: Body: žádné
Očekávaný HTTP stav: 404 Not Found
Očekávané tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
SQL dotaz v DBeaveru: SELECT * FROM public.students WHERE id = 403;
Očekávaný stav databáze: Po úspěšném DELETE student 403 neexistuje; kontrolní SELECT by vrátil 0 řádků.

TC_LOGIN_001 – Přihlášení s nesprávným heslem

Cíl: Přihlášení s nesprávným heslem.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "{{login}}",
  "password": "NespravneHeslo123"
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials", "code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_002 – Prázdné přihlašovací údaje

Cíl: Prázdné přihlašovací údaje.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
Body: Použij stejný formát jako při úspěšném přihlášení, ale obě hodnoty nech prázdné (""). Pokud používáš x-www-form-urlencoded, zachovej tento formát.
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: String should have at least 1 character; body -> password: String should have at least 6 characters","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_003 – Heslo kratší než 6 znaků

Cíl: Heslo kratší než 6 znaků.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": "{{login}}",
  "password": "12345"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> password: String should have at least 6 characters","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_004 – Heslo o délce přesně 6 znaků

Cíl: Heslo o délce přesně 6 znaků.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": "{{login}}",
  "password": "123456"
}
```
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_005 – Přihlášení bez hesla

Cíl: Přihlášení bez hesla.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "username": "test"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Field required; body -> password: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_006 – Přihlášení bez hesla se správným polem `login`

Cíl: Přihlášení bez hesla se správným polem `login`.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": "test"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> password: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_007 – Přihlášení bez uživatelského jména

Cíl: Přihlášení bez uživatelského jména.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "password": "testheslo123"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_008 – Přihlášení s prázdným uživatelským jménem

Cíl: Přihlášení s prázdným uživatelským jménem.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "",
  "password": "testheslo123"
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: String should have at least 1 character","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_009 – Přihlášení s prázdným heslem

Cíl: Přihlášení s prázdným heslem.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test",
  "password": ""
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> password: String should have at least 6 characters","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_010 – Přihlášení s loginem obsahujícím pouze mezery

Cíl: Přihlášení s loginem obsahujícím pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "   ",
  "password": "testheslo123"
}
```

Očekávaný HTTP stav: 401 Unauthorized INVALID_CREDENTIALS nebo 422 Unprocessable Entity VALIDATION_ERROR (login tvořený pouze mezerami)
Očekávané tělo odpovědi: Při 401: `{ "detail":"Invalid credentials", "code":"INVALID_CREDENTIALS" }`; při 422: JSON s `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_011 – Přihlášení s heslem obsahujícím pouze mezery

Cíl: Přihlášení s heslem obsahujícím pouze mezery.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test",
  "password": "     "
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_012 – Heslo se šesti běžnými znaky

Cíl: Heslo se šesti běžnými znaky.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test",
  "password": "abcdef"
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_013 – Přihlášení s hodnotou `null` místo hesla

Cíl: Přihlášení s hodnotou `null` místo hesla.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test",
  "password": null
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_014 – Přihlášení s hodnotou `null` místo loginu

Cíl: Přihlášení s hodnotou `null` místo loginu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": null,
  "password": "testheslo123"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_015 – Přihlášení s číselnou hodnotou loginu

Cíl: Přihlášení s číselnou hodnotou loginu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": 123456,
  "password": "testheslo123"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_016 – Přihlášení s číselnou hodnotou hesla

Cíl: Přihlášení s číselnou hodnotou hesla.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": "test",
  "password": 123456
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_017 – Přihlášení s boolean hodnotou místo hesla

Cíl: Přihlášení s boolean hodnotou místo hesla.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test",
  "password": true
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_018 – Přihlášení s boolean hodnotou místo loginu

Cíl: Přihlášení s boolean hodnotou místo loginu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": true,
  "password": "testheslo123"
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_019 – Přihlášení s neplatným JSON

Cíl: Přihlášení s neplatným JSON.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test"
  "password": "testheslo123"
}
```

Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> 24: JSON decode error","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_020 – Přihlášení s prázdným tělem požadavku

Cíl: Přihlášení s prázdným tělem požadavku.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
Tělo není odesláno: Body → none.
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_021 – Přihlášení s JSON polem místo objektu

Cíl: Přihlášení s JSON polem místo objektu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
[
  {"login": "test", "password": "testheslo123"}
]
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Input should be a valid dictionary or object to extract fields from","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_022 – Přihlášení s více neplatnými poli současně

Cíl: Přihlášení s více neplatnými poli současně.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
  {
    "login": 12345,
    "password": true
  }
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Input should be a valid string; body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_023 – Přihlášení s neočekávaným polem

Cíl: Přihlášení s neočekávaným polem.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "test",
  "password": "testheslo123",
  "isAdmin": true
}
```

Očekávaný HTTP stav: 401 Unauthorized INVALID_CREDENTIALS, pokud se extra pole ignoruje; 422 Unprocessable Entity, pokud jsou extra pole zakázaná
Očekávané tělo odpovědi: Při 401: `{ "detail":"Invalid credentials", "code":"INVALID_CREDENTIALS" }`; při 422: JSON s `code="VALIDATION_ERROR"`.
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_024 – Přihlášení s JSON hodnotou `null` místo objektu

Cíl: Přihlášení s JSON hodnotou `null` místo objektu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON: null
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_025 – Přihlášení s SQL injection v loginu

Cíl: Přihlášení s SQL injection v loginu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
  "login": "' OR '1'='1",
  "password": "testheslo123"
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_026 – Přihlášení s SQL injection v hesle

Cíl: Přihlášení s SQL injection v hesle.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": "test",
  "password": "' OR '1'='1"
}
```
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_027 – Přihlášení s prázdným JSON objektem

Cíl: Přihlášení s prázdným JSON objektem.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON: {}
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body -> login: Field required; body -> password: Field required","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_028 – Přihlášení s heslem o délce 100 znaků

Cíl: Ověřit odmítnutí neplatných přihlašovacích údajů s heslem o délce přesně 100 znaků.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": "test",
  "password":
"aaaa
```

TC_LOGIN_029 – Přihlášení s mezerami kolem loginu

Cíl: Přihlášení s mezerami kolem loginu.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
```
{
  "login": " test ",
  "password": "testheslo123"
}
```
Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_030 – Nepodporovaná HTTP metoda GET

Cíl: Nepodporovaná HTTP metoda GET.
Postup testu:
1. V Postmanu připravit požadavek GET {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).); použít ID či další parametry uvedené v URL. Tělo požadavku ponechat dle testovacích dat.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: GET {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:
Bez těla (metoda GET).
Očekávaný HTTP stav: 405 Method Not Allowed
Očekávané tělo odpovědi: {"detail": "Method Not Allowed","code": "UNEXPECTED_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_031 – Přihlášení s nepodporovaným Content-Type

Cíl: Přihlášení s nepodporovaným Content-Type.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. V Postmanu zvolit No Auth, v Body → raw vybrat Text, nastavit hlavičku Content-Type: text/plain a vložit níže uvedené testovací údaje jako text.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / raw text (Content-Type: text/plain):
```
{
  "login": "test",
  "password": "testheslo123"
}
```
Očekávaný HTTP stav: 422 Unprocessable Entity
Očekávané tělo odpovědi: {"detail": "body: Input should be a valid dictionary or object to extract fields from","code": "VALIDATION_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_032 – Přihlášení s Unicode znaky

Cíl: Přihlášení s Unicode znaky.
Postup testu:
1. V Postmanu připravit požadavek POST {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: POST {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
 "login": "PřílišŽluťoučký_测试_😀",
 "password": "testheslo123"
}
```

Očekávaný HTTP stav: 401 Unauthorized
Očekávané tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

TC_LOGIN_033 – Přihlášení s nesprávnou HTTP metodou PUT

Cíl: Přihlášení s nesprávnou HTTP metodou PUT.
Postup testu:
1. V Postmanu připravit požadavek PUT {{base_url}}/login.
2. Nastavit autorizaci (Bez autorizace (No Auth).) a hlavičku Content-Type: application/json; do Body → raw vložit níže uvedená testovací data.
3. Odeslat požadavek; ověřit HTTP stavový kód a JSON odpovědi proti očekávanému výsledku.
4. Pokud je vyžadována kontrola vedlejších účinků, porovnat počet studentů před a po požadavku uvedeným SQL dotazem. Tento dotaz netestuje správnost přihlášení.

Metoda a URL: PUT {{base_url}}/login
Autorizace: Bez autorizace (No Auth).
Testovací data / JSON:

```
{
 "login": "test",
 "password": "testheslo123"
}
```

Očekávaný HTTP stav: 405 Method Not Allowed
Očekávané tělo odpovědi: {"detail": "Method Not Allowed","code": "UNEXPECTED_ERROR"}
SQL dotaz v DBeaveru: SELECT COUNT(*) AS pocet_studentu FROM public.students;
Očekávaný stav databáze: Bez změny v tabulce studentů; jde o test autentizace.

EXEKUCE TESTŮ

TC_GET_001 – Načtení existujícího studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 376, "firstName": "TestJan", "lastName": "Novak", "email": "testjan.novak.20261029@example.com", "isEuCitizen": true, "numberOfFailedStudies": 0}
Kontrola databáze: GET úspěšně načetl studenta ID 376 (HTTP 200 OK). Jedná se o čtení bez zápisu do databáze.
Výsledek: PASS

TC_GET_002 – Neexistující student

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: SELECT nevrátil žádný řádek; hledaný student v databázi neexistuje.
Výsledek: FAIL
Související chyba: BUG_001

TC_GET_003 – Neplatný datový typ ID

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: GET skončil HTTP 422 Unprocessable Entity; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_004 – Chybějící autorizace

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: GET skončil HTTP 403 Forbidden; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: FAIL

TC_GET_005 – Neplatný token

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: GET skončil HTTP 401 Unauthorized; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_006 – Záporné ID studenta

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: Původní kontrola databáze potvrdila, že hledaný záznam neexistuje.
Výsledek: FAIL
Související chyba: BUG_001

TC_GET_007 – Ověření PSČ přes GET

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestPSCPlatne","lastName": "STUDENT","email": "test.psc.platne.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: Hodnota PSČ nebyla v databázi uložena.
Výsledek: PASS

TC_GET_008 – Ověření nově vytvořeného studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 398, "email": "TEST.VELKA.PISMENA.20261029@EXAMPLE.COM"}
Kontrola databáze: DBeaver potvrdil shodu GET odpovědi a záznamu studenta ID 398. GET neprovádí zápis.
Výsledek: PASS

TC_GET_009 – Přístup ke studentovi pomocí desetinného ID

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: API správně odmítlo desetinné ID 403.5 a původní záznam v databázi zůstal beze změny.
Výsledek: PASS

TC_GET_010 – ID s nulami na začátku

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: GET úspěšně načetl studenta ID 403 (e-mail "test.id.text.20261029@example.com") (HTTP 200 OK). Jedná se o čtení bez zápisu do databáze.
Výsledek: PASS

TC_GET_011 – Překročení maximálního rozsahu ID

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: GET skončil HTTP 500 Internal Server Error; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: FAIL
Související chyba: BUG_007

TC_GET_012 – ID na maximální hranici integeru

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: GET skončil HTTP 500 Internal Server Error; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: FAIL
Související chyba: BUG_001

TC_GET_013 – ID obsahující pouze mezery

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: GET skončil HTTP 422 Unprocessable Entity; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_014 – ID s kladným znaménkem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: GET úspěšně načetl studenta ID 403 (e-mail "test.id.text.20261029@example.com") (HTTP 200 OK). Jedná se o čtení bez zápisu do databáze.
Výsledek: PASS

TC_GET_015 – ID s desetinnou nulou

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: GET vrátil 200 OK a načetl studenta 403 i pro cestu /students/403.0; databáze zůstala beze změny.
Výsledek: PASS

TC_GET_016 – ID ve vědeckém zápisu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: GET skončil HTTP 422 Unprocessable Entity; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_017 – SQL injection v identifikátoru studenta

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: GET skončil HTTP 422 Unprocessable Entity; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_018 – HTML/JavaScript v identifikátoru

Skutečný HTTP stav: 404 Not Found
Skutečné tělo odpovědi: {"detail": "Not Found","code": "NOT_FOUND"}
Kontrola databáze: GET skončil HTTP 404 Not Found; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_019 – Neplatný Bearer token

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: GET skončil HTTP 403 Forbidden; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: FAIL

TC_GET_020 – Neplatný token ve správném formátu JWT

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: GET skončil HTTP 401 Unauthorized; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_021 – Přístup s tokenem bez schématu Bearer

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: GET skončil HTTP 403 Forbidden; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_022 – Nesprávné autorizační schéma Basic

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Invalid authentication credentials","code": "FORBIDDEN"}
Kontrola databáze: GET skončil HTTP 403 Forbidden; požadované údaje nebyly úspěšně načteny. Čtecí operace nemění záznamy v databázi.
Výsledek: PASS

TC_GET_023 – Platný token s nesprávně napsaným schématem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: GET úspěšně načetl studenta ID 403 (e-mail "test.id.text.20261029@example.com") (HTTP 200 OK). Jedná se o čtení bez zápisu do databáze.
Výsledek: PASS

TC_POST_001 – Vytvoření studenta s platnými údaji

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 376, "firstName": "TestJan", "lastName": "Novak", "email": "testjan.novak.20261029@example.com", "isEuCitizen": true, "numberOfFailedStudies": 0}
Kontrola databáze: Student s ID 376 byl nalezen v databázi.
Výsledek: PASS

TC_POST_002 – Vytvoření studenta s prázdným JSON

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 377,"firstName": null,"lastName": null,"email": null,"isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: Údaje odpovídaly záznamu studenta s ID 377.
Výsledek: PASS

TC_POST_003 – Neplatný počet neúspěšných studií

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
Kontrola databáze: POST odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo vytvoření nového studenta.
Výsledek: PASS

TC_POST_004 – Neplatný datový typ

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: POST odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo vytvoření nového studenta.
Výsledek: PASS

TC_POST_005 – Příliš dlouhé jméno

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> firstName: String should have at most 255 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: POST odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo vytvoření nového studenta.
Výsledek: PASS

TC_POST_006 – Jméno o délce přesně 255 znaků

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 378, "firstName": "AAAA

TC_POST_010 – Neplatný datový typ boolean

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> isEuCitizen: Input should be a valid boolean, unable to interpret input","code": "VALIDATION_ERROR"}
Kontrola databáze: záznam nevznikl
Výsledek: PASS

TC_POST_011 – Neplatné PSČ

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 380,"firstName": "TestPSC","lastName": "NEPLATNE","email": "test.psc.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: V databázi vznikl student ID 380 (HTTP 201 Created); hodnota zipCode="ABCDE" se do databázového pole PSČ neuložila.
Výsledek: PASS

TC_POST_012 – Nepovolené pole v JSON

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 381,"firstName": "TestExtra","lastName": "POLE","email": "test.extra.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: shoduje se
Výsledek: PASS

TC_POST_013 – Uložení platného PSČ při vytvoření

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestPSCPlatne","lastName": "STUDENT","email": "test.psc.platne.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: Student 382 vznikl, ale zip_code se v DB neuložil; potvrzeno screenshotem DBeaveru.
Výsledek: FAIL

TC_POST_014 – Neplatný datový typ jména firstName

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string", "code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_015 – Neplatný typ příjmení

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> lastName: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_016 – Neplatný typ e-mailu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> email: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_017 – E-mail delší než 255 znaků

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> email: String should have at most 255 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_018 – E-mail přesně 255 znaků

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 383,"firstName": "TestEmail255","lastName": "LIMIT255","email": "bbbb

TC_POST_025 – Boolean místo počtu neúspěšných studií

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 393,"firstName": "TestBooleanStudium","lastName": "BOOLEAN","email": "test.boolean.studium.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 1}
Kontrola databáze: API odpovědělo HTTP 201 Created a vrátilo vytvořeného studenta ID 393, e-mail "test.boolean.studium.20261029@example.com". SQL dotaz je zaměřen na ověření tohoto záznamu.
Výsledek: FAIL

TC_POST_026 – Boolean `false` místo počtu studií

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 394,"firstName": "TestBooleanFalse","lastName": "STUDIUM","email": "test.boolean.false.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: API odpovědělo HTTP 201 Created a vrátilo vytvořeného studenta ID 394, e-mail "test.boolean.false.20261029@example.com". SQL dotaz je zaměřen na ověření tohoto záznamu.
Výsledek: FAIL

TC_POST_027 – Číslo zadané jako text

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 395,"firstName": "TestCisloText","lastName": "STUDIUM","email": "test.cislo.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 3}
Kontrola databáze: API odpovědělo HTTP 201 Created a vrátilo vytvořeného studenta ID 395, e-mail "test.cislo.text.20261029@example.com". SQL dotaz je zaměřen na ověření tohoto záznamu.
Výsledek: PASS

TC_POST_028 – Desetinné číslo zadané jako text

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_029 – Příliš vysoký počet neúspěšných studií

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: FAIL
Související chyba: BUG_006

TC_POST_030 – Maximální hodnota 32bitového integeru

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 396,"firstName": "TestMaxInteger","lastName": "STUDIUM","email": "test.max.integer.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 2147483647}
Kontrola databáze: HTTP 201 Created; odpověď API obsahuje vytvořeného studenta ID 396. Kontrolní SQL SELECT se vztahuje ke stejnému ID.
Výsledek: PASS

TC_POST_031 – Velké číslo zadané jako text

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: FAIL
Související chyba: BUG_006

TC_POST_032 – Velmi dlouhé příjmení

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> lastName: String should have at most 255 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_033 – Příjmení přesně 255 znaků

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 397,"firstName": "TestPrijmeni255","lastName":
"NNNN

TC_POST_040 – Záporné ID zadané jako text

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "test.id.text.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: HTTP 201 Created; odpověď API obsahuje vytvořeného studenta ID 403. Kontrolní SQL SELECT se vztahuje ke stejnému ID.
Výsledek: PASS

TC_POST_041 – Hodnota `null` v poli `email`

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 404,"firstName": "TestNullEmailPost","lastName": "STUDENT","email": null,"isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: HTTP 201 Created; odpověď API obsahuje vytvořeného studenta ID 404. Kontrolní SQL SELECT se vztahuje ke stejnému ID.
Výsledek: PASS

TC_POST_042 – Hodnota `null` v poli `numberOfFailedStudies`

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 405,"firstName": "TestNullFailedStudies","lastName": "STUDENT","email": "test.null.failed.studies.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: Vytvoření záznamu ID 405 (firstName="TestNullFailedStudies", lastName="STUDENT") bylo potvrzeno i zápisem v databázi.
Výsledek: PASS

TC_POST_043 – Hodnota `null` v poli `isEuCitizen`

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 406,"firstName": "TestNullEuCitizen","lastName": "STUDENT","email": "test.null.eu.citizen.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: Vytvoření záznamu ID 406 (firstName="TestNullEuCitizen", lastName="STUDENT") bylo potvrzeno i zápisem v databázi.
Výsledek: PASS

TC_POST_044 – Neplatný datový typ `firstName` – pole

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_045 – Neplatný datový typ `lastName` – objekt

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> lastName: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi nevznikl nový záznam.
Výsledek: PASS

TC_POST_046 – Prázdný řetězec v poli `lastName`

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 407,"firstName": "TestPrazdnePrijmeni","lastName": null,"email": "test.prazdne.prijmeni.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: Vytvoření záznamu ID 407 (firstName="TestPrazdnePrijmeni", lastName=null) bylo potvrzeno i zápisem v databázi.
Výsledek: PASS

TC_POST_047 – Prázdný řetězec v poli `firstName`

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 408,"firstName": "","lastName": "TESTPRAZDNEJMENO","email": "test.prazdne.jmeno.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: Vytvoření záznamu ID 408 (firstName="", lastName="TESTPRAZDNEJMENO") bylo potvrzeno i zápisem v databázi.
Výsledek: PASS

TC_POST_048 – Prázdný řetězec v poli `email`

Skutečný HTTP stav: 201 Created
Skutečné tělo odpovědi:
{"id": 409,"firstName": "TestPrazdnyEmail","lastName": "STUDENT","email": "","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: Vytvoření záznamu ID 409 (firstName="TestPrazdnyEmail", lastName="STUDENT") bylo potvrzeno i zápisem v databázi.
Výsledek: PASS

TC_POST_049 – Neplatný typ `isEuCitizen` – JSON objekt

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean","code": "VALIDATION_ERROR"}
Kontrola databáze: POST odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo vytvoření nového studenta.
Výsledek: PASS

TC_POST_050 – Duplicitní e-mail s `null` jménem

Skutečný HTTP stav: 400 Bad Request
Skutečné tělo odpovědi: {"detail": "Student with the same e-mail already exists","code": "STUDENT_EMAIL_EXISTS"}
Kontrola databáze: POST odmítnut (HTTP 400 Bad Request); API nepotvrdilo vytvoření nového studenta.
Výsledek: PASS

TC_POST_051 – Neplatný typ `numberOfFailedStudies` – JSON pole

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be a valid integer","code": "VALIDATION_ERROR"}
Kontrola databáze: POST odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo vytvoření nového studenta.
Výsledek: PASS

TC_PUT_001 – Úprava existujícího studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 376, "firstName": "JanUpraveny", "lastName": "Novak", "email": "testjan.novak.20261029@example.com", "isEuCitizen": true, "numberOfFailedStudies": 1}
Kontrola databáze: V DBeaveru bylo potvrzeno uložení změny number_of_failed_studies z 0 na 1 u studenta 376.
Výsledek: PASS

TC_PUT_002 – Částečná aktualizace studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 376,"firstName": "JanUpraveny","lastName": "Novak","email": "testjan.novak.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 2}
Kontrola databáze: DBeaver: id=376; number_of_failed_studies=2; is_eu_citizen=true. Ostatní údaje zůstaly zachované (potvrzeno).
Výsledek: PASS

TC_PUT_003 – Záporný počet neúspěšných studií

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
Kontrola databáze: V databázi zůstala původní hodnota numberOfFailedStudies = 2.
Výsledek: PASS

TC_PUT_004 – Aktualizace neexistujícího studenta

Skutečný HTTP stav: 404 Not Found
Skutečné tělo odpovědi: {"detail": "Chyba, nelze zmenit data neexistujuceho studenta","code": "NOT_FOUND"}
Kontrola databáze: PUT byl odmítnut (HTTP 404 Not Found); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_005 – Neplatný datový typ

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: pořád 2
Výsledek: PASS

TC_PUT_006 – Překročení maximální délky jména

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> firstName: String should have at most 255 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: zustalo to tak
Výsledek: PASS

TC_PUT_007 – Jméno o délce 255 znaků

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 376, "firstName":
"AAAA

TC_PUT_014 – Nechtěná změna `isEuCitizen`

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 377, "firstName": "TestAktualizace", "isEuCitizen": false}
Kontrola databáze: U studenta 377 se změnilo jméno na TestAktualizace a současně se is_eu_citizen nesprávně změnilo z true na false. Potvrzeno v DBeaveru.
Výsledek: FAIL
Související chyba: BUG_005

TC_PUT_015 – Přepis dalších neuvedených polí

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 377,"firstName": "TestZachovaniHodnot","lastName": null,"email": "neplatny-email-put","isEuCitizen": false,"numberOfFailedStudies": 3}
Kontrola databáze: Změna jména na TestZachovaniHodnot potvrzena; numberOfFailedStudies zůstalo 3, ale isEuCitizen se nesprávně změnilo z true na false.
Výsledek: FAIL
Související chyba: BUG_005

TC_PUT_016 – Aktualizace pouze počtu neúspěšných studií

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 377, "isEuCitizen": false, "numberOfFailedStudies": 4}
Kontrola databáze: DBeaver potvrdil změnu number_of_failed_studies z 3 na 4 a nechtěnou změnu is_eu_citizen z true na false.
Výsledek: FAIL
Související chyba: BUG_005

TC_PUT_017 – Zachování PSČ při aktualizaci

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 377,"firstName": "TestZachovaniHodnot","lastName": null,"email": "neplatny-email-put","isEuCitizen": false,"numberOfFailedStudies": 4}
Kontrola databáze: Databázový záznam se nezměnil.
Výsledek: FAIL

TC_PUT_018 – Zachování příjmení při změně jména

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestZmenyJmena","lastName": "STUDENT","email": "test.psc.platne.20261029@example.com","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=382; first_name="TestZmenyJmena"; last_name="STUDENT"; is_eu_citizen=false; number_of_failed_studies=0.
Výsledek: FAIL
Související chyba: BUG_005

TC_PUT_019 – Nastavení příjmení na NULL

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body: Value error, At least one field must be provided for update","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: FAIL

TC_PUT_020 – PUT s hodnotou null v poli lastName

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestNullPrijmeni","lastName": "STUDENT","email": "test.psc.platne.20261029@example.com","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: Ověření v databázi potvrdilo shodu uložených hodnot s odpovědí API.
Výsledek: FAIL
Související chyba: BUG_005

TC_PUT_021 – Prázdný e-mail

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestNullPrijmeni","lastName": "STUDENT","email": "","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: E-mail je v databázi uložený jako prázdný řetězec.
Výsledek: FAIL
Související chyba: BUG_005

TC_PUT_022 – Příliš dlouhý e-mail

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> email: String should have at most 255 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_023 – E-mail přesně 255 znaků

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestNullPrijmeni","lastName": "STUDENT","email": "aaaa

TC_PUT_028 – Aktualizace na stejnou hodnotu

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestNullBoolean","lastName": "STUDENT","email":
"aaaa

TC_PUT_034 – Aktualizace více polí současně

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 382,"firstName": "TestVicePoli","lastName": "Aktualizovany","email":
"aaaa

TC_PUT_042 – Překročení maximální hodnoty celého čísla

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: nic se nezměnilo
Výsledek: FAIL
Související chyba: BUG_006

TC_PUT_043 – Neplatný datový typ počtu studií

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_044 – Desetinné číslo v počtu neúspěšných studií

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> numberOfFailedStudies: Input should be a valid integer, got a number with a fractional part","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_045 – Neplatný datový typ `isEuCitizen`

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> isEuCitizen: Input should be a valid boolean, unable to interpret input","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_046 – Aktualizace s prázdným JSON objektem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body: Value error, At least one field must be provided for update","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_047 – Aktualizace s neplatným JSON

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> 23: JSON decode error","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_048 – Aktualizace s JSON polem místo objektu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body: Input should be a valid dictionary or object to extract fields from","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_049 – Aktualizace s duplicitním e-mailem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "TEST.VELKA.PISMENA.20261029@EXAMPLE.COM","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: PUT vrátil 200 OK a studentovi 403 změnil e-mail na hodnotu již používanou studentem 398; požadavek neměl být přijat.
Výsledek: FAIL
Související chyba: BUG_004, BUG_005

TC_PUT_050 – E-mail s neplatným formátem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "neplatny-email-bez-zavinace","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver potvrdil u studenta ID 403 uložení neplatného e-mailu „neplatny-email-bez-zavinace“ při HTTP 200 OK.
Výsledek: FAIL

TC_PUT_051 – E-mail obsahující pouze mezery

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "   ","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver potvrdil u studenta ID 403 uložení e-mailu obsahujícího pouze tři mezery při HTTP 200 OK.
Výsledek: FAIL

TC_PUT_052 – Aktualizace e-mailu na hodnotu `null`

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body: Value error, At least one field must be provided for update","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_053 – Aktualizace s hodnotou `true` místo čísla

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "   ","isEuCitizen": false,"numberOfFailedStudies": 1}
Kontrola databáze: V databázi je uložena hodnota 1.
Výsledek: FAIL

TC_PUT_054 – Aktualizace čísla pomocí textové hodnoty

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "   ","isEuCitizen": true,"numberOfFailedStudies": 3}
Kontrola databáze: Hodnoty v databázi byly aktualizovány.
Výsledek: PASS

TC_PUT_055 – Překročení rozsahu čísla zadaného jako text

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: Student 403 zůstal v databázi beze změny; příliš vysoká hodnota numberOfFailedStudies nebyla uložena.
Výsledek: FAIL
Související chyba: BUG_006

TC_PUT_056 – Aktualizace s nulovou hodnotou

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestIdText","lastName": "STUDENT","email": "   ","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; number_of_failed_studies=0 (potvrzené uložení).
Výsledek: PASS

TC_PUT_057 – Aktualizace s ID obsahujícím počáteční nuly

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestPocatecniNuly","lastName": "STUDENT","email": "   ","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="TestPocatecniNuly". Zadané ID 000403 se vztahovalo k záznamu 403.
Výsledek: PASS

TC_PUT_058 – Aktualizace s desetinným zápisem celého ID

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestDesetinneID","lastName": "STUDENT","email": "   ","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="TestDesetinneID". Zadané ID 403.0 se vztahovalo k záznamu 403.
Výsledek: PASS

TC_PUT_059 – Aktualizace s ID mimo rozsah 32bitového integeru

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: Databáze zůstala beze změny; hodnota TestVelkeID nebyla zapsána.
Výsledek: FAIL
Související chyba: BUG_007

TC_PUT_060 – Aktualizace s neplatným textovým ID

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_061 – Aktualizace s negativním ID

Skutečný HTTP stav: 404 Not Found
Skutečné tělo odpovědi: {"detail": "Chyba, nelze zmenit data neexistujuceho studenta","code": "NOT_FOUND"}
Kontrola databáze: PUT byl odmítnut (HTTP 404 Not Found); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_062 – Aktualizace s ID obsahujícím mezeru

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_063 – Aktualizace s ID obsahujícím SQL výraz

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_064 – Aktualizace s neznámým polem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestNeznamePole","lastName": "STUDENT","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="TestNeznamePole" (uložení potvrzeno); neznámé pole favoriteColor bylo ignorováno.
Výsledek: PASS

TC_PUT_065 – Aktualizace s pokusem změnit ID studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestZmenaID","lastName": "STUDENT","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="TestZmenaID"; ID studenta zůstalo 403.
Výsledek: PASS

TC_PUT_066 – Aktualizace s prázdným jménem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "","lastName": "STUDENT","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="".
Výsledek: PASS

TC_PUT_067 – Aktualizace s prázdným příjmením

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "","lastName": "","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; last_name="".
Výsledek: PASS

TC_PUT_068 – Aktualizace jména obsahujícího pouze mezery

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": " ","lastName": "","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name=" ".
Výsledek: PASS

TC_PUT_069 – Aktualizace jména s českou diakritikou

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "Jiří","lastName": "Dvořáková","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="Jiří"; last_name="Dvořáková" (uložení obou hodnot potvrzeno).
Výsledek: PASS

TC_PUT_070 – Aktualizace jména pomocí hodnoty `false`

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> firstName: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Podle závěru původní exekuce zůstal záznam beze změny.
Výsledek: PASS

TC_PUT_071 – Aktualizace příjmení pomocí pole

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> lastName: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Podle závěru původní exekuce zůstal záznam beze změny.
Výsledek: PASS

TC_PUT_072 – Aktualizace s neplatným datovým typem e-mailu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> email: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Podle závěru původní exekuce zůstal záznam beze změny.
Výsledek: PASS

TC_PUT_073 – Aktualizace s hodnotou `isEuCitizen` jako pole

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> isEuCitizen: Input should be a valid boolean","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_074 – Aktualizace s prázdným textovým tělem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_075 – Aktualizace bez autentizačního tokenu

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: PUT byl odmítnut (HTTP 403 Forbidden); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_076 – Aktualizace s neplatným tokenem

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: PUT byl odmítnut (HTTP 401 Unauthorized); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_077 – Aktualizace s prošlým tokenem

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi:
{"detail": "Invalid token", "code": "INVALID_TOKEN"}
Kontrola databáze: PUT byl odmítnut (HTTP 401 Unauthorized); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_078 – Aktualizace s hodnotou `null` v JSON těle

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_079 – Aktualizace s více neplatnými poli současně

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> firstName: Input should be a valid string; body -> lastName: Input should be a valid string; body -> email: Input should be a valid string; body -> isEuCitizen: Input should be a valid boolean, unable to interpret input; body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_080 – Aktualizace s neplatným JSON a chybějícím tokenem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> 23: JSON decode error","code": "VALIDATION_ERROR"}
Kontrola databáze: Podle závěru původní exekuce zůstal záznam beze změny.
Výsledek: PASS

TC_PUT_081 – Neplatný token a platné aktualizační údaje

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: PUT byl odmítnut (HTTP 401 Unauthorized); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_082 – Aktualizace s neplatným tokenem a neexistujícím studentem

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: PUT byl odmítnut (HTTP 401 Unauthorized); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_083 – Neexistující student a neplatná data

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> firstName: Input should be a valid string; body -> numberOfFailedStudies: Input should be greater than or equal to 0","code": "VALIDATION_ERROR"}
Kontrola databáze: PUT byl odmítnut (HTTP 422 Unprocessable Entity); API nepotvrdilo provedení aktualizace.
Výsledek: PASS

TC_PUT_084 – Aktualizace s duplicitním polem v JSON

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "DruhaHodnota","lastName": "Dvořáková","email": " ","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; first_name="DruhaHodnota" (uložena druhá hodnota duplicitního JSON klíče).
Výsledek: PASS

TC_PUT_085 – Aktualizace s hodnotou `isEuCitizen` jako text

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "DruhaHodnota","lastName": "Dvořáková","email": " ","isEuCitizen": false,"numberOfFailedStudies": 0}
Kontrola databáze: DBeaver: id=403; is_eu_citizen=false, ačkoli byl odeslán textový řetězec "true".
Výsledek: FAIL

TC_PUT_086 – Aktualizace e-mailu na maximální délku 255 znaků

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "DruhaHodnota","lastName": "Dvořáková","email":
"aaaa

TC_PUT_091 – Aktualizace více platných polí současně

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "Petr","lastName": "Novák","email": "petr.novak.test092@example.com","isEuCitizen": false,"numberOfFailedStudies": 2}
Kontrola databáze: DBeaver: id=403; first_name="Petr"; last_name="Novák"; email="petr.novak.test092@example.com"; number_of_failed_studies=2. Všechna čtyři pole potvrzena.
Výsledek: PASS

TC_PUT_092 – Aktualizace se současně platným a neplatným e-mailem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> email: String should have at most 255 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: Podle závěru původní exekuce zůstal záznam beze změny.
Výsledek: PASS

TC_PUT_093 – Aktualizace e-mailu na hodnotu `null` společně s platným jménem

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "TestNullEmail","lastName": "Novák","email": "petr.novak.test092@example.com","isEuCitizen": true,"numberOfFailedStudies": 2}
Kontrola databáze: DBeaver: id=403; first_name="TestNullEmail"; email="petr.novak.test092@example.com" (e-mail zůstal nezměněn).
Výsledek: PASS

TC_PUT_094 – Aktualizace pomocí JSON s prázdným řetězcem a platnou hodnotou

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "","lastName": "TestPrazdneJmeno","email": "petr.novak.test092@example.com","isEuCitizen": false,"numberOfFailedStudies": 2}
Kontrola databáze: DBeaver: id=403; first_name=""; last_name="TestPrazdneJmeno" (uložení obou změn potvrzeno).
Výsledek: PASS

TC_PUT_095 – Aktualizace se stejnými hodnotami

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "","lastName": "TestPrazdneJmeno","email": "petr.novak.test092@example.com","isEuCitizen": true,"numberOfFailedStudies": 2}
Kontrola databáze: DBeaver: id=403; first_name=""; last_name="TestPrazdneJmeno"; email="petr.novak.test092@example.com"; number_of_failed_studies=2. Testovaná pole beze změny.
Výsledek: PASS

TC_DELETE_001 – Smazání existujícího studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: Původní student s ID 379 zůstal v databázi i po opakovaném požadavku DELETE (HTTP 200 OK).
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_002 – Kontrola dostupnosti studenta po DELETE

Skutečný HTTP stav: 200 OK (DELETE), 200 OK (následný GET)
Skutečné tělo odpovědi:
{"id": 379,"firstName": "TestEmail","lastName": "NEPLATNY","email": "neplatny-email","isEuCitizen": true,"numberOfFailedStudies": 0}
Kontrola databáze: Student ID 379 po požadavku zůstal dostupný (GET 200); při navazující kontrole se potvrdilo, že smazání nebylo provedeno.
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_003 – Opakované smazání

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: Student ID 379 zůstal uložený; opakovaná operace přesto vrátila odpověď o úspěšném smazání.
Výsledek: FAIL

Související chyba: BUG_002

TC_DELETE_004 – Smazání neexistujícího studenta

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: DELETE skončil HTTP 500 Internal Server Error; API nepotvrdilo odstranění záznamu.
Výsledek: FAIL
Související chyba: BUG_003

TC_DELETE_005 – Neplatný formát ID

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_006 – Smazání bez autentizace

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: DELETE skončil HTTP 403 Forbidden; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_007 – Smazání s neplatným tokenem

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: Student zůstal uložen v databázi; požadavek s neplatným tokenem záznam neodstranil.
Výsledek: PASS

TC_DELETE_008 – Smazání existujícího studenta a ověření v databázi

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: Po DELETE /students/403 vrátil kontrolní SELECT * FROM public.students WHERE id = 403 stále záznam studenta ID 403. API přesto vrátilo 200 OK a zprávu o úspěšném smazání. Potvrzen BUG_002.
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_009 – Smazání studenta s desetinným ID

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_010 – Překročení maximální hodnoty ID

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Internal server error","code": "INTERNAL_SERVER_ERROR"}
Kontrola databáze: DELETE skončil HTTP 500 Internal Server Error; API nepotvrdilo odstranění záznamu.
Výsledek: FAIL
Související chyba: BUG_007

TC_DELETE_011 – Smazání studenta se záporným ID

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: DELETE skončil HTTP 500 Internal Server Error; API nepotvrdilo odstranění záznamu.
Výsledek: FAIL
Související chyba: BUG_003

TC_DELETE_012 – Smazání studenta s ID obsahujícím počáteční nuly

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: Databázový záznam zůstal beze změny.
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_013 – Smazání studenta s neplatným tokenem a neexistujícím ID

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: DELETE skončil HTTP 500 Internal Server Error; API nepotvrdilo odstranění záznamu.
Výsledek: FAIL

TC_DELETE_014 – Neplatný token a existující student

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token", "code": "INVALID_TOKEN"}
Kontrola databáze: Požadavek s neplatným tokenem byl odmítnut (HTTP 401 Unauthorized); DELETE neprovedl odstranění studenta 403.
Výsledek: PASS

TC_DELETE_015 – Chybějící token a existující student

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: DELETE skončil HTTP 403 Forbidden; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_016 – Chybějící token a neexistující student

Skutečný HTTP stav: 403 Forbidden
Skutečné tělo odpovědi: {"detail": "Not authenticated","code": "FORBIDDEN"}
Kontrola databáze: DELETE skončil HTTP 403 Forbidden; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_017 – Neplatný token a neexistující student s jiným ID

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: DELETE skončil HTTP 401 Unauthorized; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_018 – Opakování TC_DELETE_013

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: DELETE skončil HTTP 401 Unauthorized; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_019 – Smazání studenta s ID ve formátu `403.0`

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: API ohlásilo smazání studenta ID 403 (HTTP 200 OK). U této operace je rozhodující následný SELECT; aplikace v dalších testech vykazovala problém s fyzickým mazáním záznamů.
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_020 – SQL injection v ID studenta

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_021 – Smazání s prázdným ID

Skutečný HTTP stav: 405 Method Not Allowed
Skutečné tělo odpovědi: {"detail": "Method Not Allowed","code": "UNEXPECTED_ERROR"}
Kontrola databáze: DELETE skončil HTTP 405 Method Not Allowed; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_022 – Smazání studenta s ID obsahujícím písmena

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_023 – Smazání s ID obsahujícím mezery

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_024 – Smazání studenta s ID začínajícím znaménkem plus

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: Databázový záznam zůstal beze změny.
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_025 – Neplatný token a neplatné ID

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid token","code": "INVALID_TOKEN"}
Kontrola databáze: DELETE skončil HTTP 401 Unauthorized; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_026 – Platný token a ID s nulou

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: DELETE skončil HTTP 500 Internal Server Error; API nepotvrdilo odstranění záznamu.
Výsledek: FAIL
Související chyba: BUG_003

TC_DELETE_027 – Smazání studenta s ID obsahujícím desetinnou čárku

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_028 – Smazání studenta s ID ve vědeckém zápisu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "path -> student_id: Input should be a valid integer, unable to parse string as an integer","code": "VALIDATION_ERROR"}
Kontrola databáze: DELETE skončil HTTP 422 Unprocessable Entity; API nepotvrdilo odstranění záznamu.
Výsledek: PASS

TC_DELETE_029 – Neexistující student s maximálním 32bitovým ID

Skutečný HTTP stav: 500 Internal Server Error
Skutečné tělo odpovědi: {"detail": "Chyba, nelze najit studenta","code": "NOT_FOUND"}
Kontrola databáze: DELETE skončil HTTP 500 Internal Server Error; API nepotvrdilo odstranění záznamu.
Výsledek: FAIL
Související chyba: BUG_003

TC_DELETE_030 – Opakované smazání stejného studenta

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi: {"message": "Student deleted successfully"}
Kontrola databáze: Student 403 v databázi zůstal; opakované DELETE záznam neodstranilo, ačkoli API vrátilo 200.
Výsledek: FAIL
Související chyba: BUG_002

TC_DELETE_031 – Ověření existence studenta po smazání pomocí GET

Skutečný HTTP stav: 200 OK
Skutečné tělo odpovědi:
{"id": 403,"firstName": "","lastName": "TestPrazdneJmeno","email": "petr.novak.test092@example.com","isEuCitizen": true,"numberOfFailedStudies": 2}
Kontrola databáze: Následný GET po DELETE vrátil 200 OK a kompletního studenta 403; student tedy zůstává dostupný i po ohlášeném smazání.
Výsledek: FAIL
Související chyba: BUG_002

TC_LOGIN_001 – Přihlášení s nesprávným heslem

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials", "code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_002 – Prázdné přihlašovací údaje

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> login: String should have at least 1 character; body -> password: String should have at least 6 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_003 – Heslo kratší než 6 znaků

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> password: String should have at least 6 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_004 – Heslo o délce přesně 6 znaků

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi:
{"detail": "Invalid credentials", "code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_005 – Přihlášení bez hesla

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> login: Field required; body -> password: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_006 – Přihlášení bez hesla se správným polem `login`

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> password: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_007 – Přihlášení bez uživatelského jména

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> login: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_008 – Přihlášení s prázdným uživatelským jménem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> login: String should have at least 1 character","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_009 – Přihlášení s prázdným heslem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> password: String should have at least 6 characters","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_010 – Přihlášení s loginem obsahujícím pouze mezery

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_011 – Přihlášení s heslem obsahujícím pouze mezery

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_012 – Heslo se šesti běžnými znaky

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_013 – Přihlášení s hodnotou `null` místo hesla

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_014 – Přihlášení s hodnotou `null` místo loginu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> login: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_015 – Přihlášení s číselnou hodnotou loginu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> login: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_016 – Přihlášení s číselnou hodnotou hesla

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_017 – Přihlášení s boolean hodnotou místo hesla

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_018 – Přihlášení s boolean hodnotou místo loginu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> login: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_019 – Přihlášení s neplatným JSON

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body -> 24: JSON decode error","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_020 – Přihlášení s prázdným tělem požadavku

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_021 – Přihlášení s JSON polem místo objektu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body: Input should be a valid dictionary or object to extract fields from","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_022 – Přihlášení s více neplatnými poli současně

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> login: Input should be a valid string; body -> password: Input should be a valid string","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_023 – Přihlášení s neočekávaným polem

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_024 – Přihlášení s JSON hodnotou `null` místo objektu

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi: {"detail": "body: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_025 – Přihlášení s SQL injection v loginu

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_026 – Přihlášení s SQL injection v hesle

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_027 – Přihlášení s prázdným JSON objektem

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body -> login: Field required; body -> password: Field required","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_028 – Přihlášení s heslem o délce 100 znaků

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials", "code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_029 – Přihlášení s mezerami kolem loginu

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_030 – Nepodporovaná HTTP metoda GET

Skutečný HTTP stav: 405 Method Not Allowed
Skutečné tělo odpovědi: {"detail": "Method Not Allowed","code": "UNEXPECTED_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 405 Method Not Allowed. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_031 – Přihlášení s nepodporovaným Content-Type

Skutečný HTTP stav: 422 Unprocessable Entity
Skutečné tělo odpovědi:
{"detail": "body: Input should be a valid dictionary or object to extract fields from","code": "VALIDATION_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 422 Unprocessable Entity. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_032 – Přihlášení s Unicode znaky

Skutečný HTTP stav: 401 Unauthorized
Skutečné tělo odpovědi: {"detail": "Invalid credentials","code": "INVALID_CREDENTIALS"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 401 Unauthorized. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

TC_LOGIN_033 – Přihlášení s nesprávnou HTTP metodou PUT

Skutečný HTTP stav: 405 Method Not Allowed
Skutečné tělo odpovědi: {"detail": "Method Not Allowed","code": "UNEXPECTED_ERROR"}
Kontrola databáze: Přihlašovací endpoint vrátil HTTP 405 Method Not Allowed. Operace nepracuje se zápisem do tabulky public.students.
Výsledek: PASS

BUG REPORT

Na základě provedených scénářů byly identifikovány následující chyby aplikace.

bug	probability	severity	test case number	mitigace
BUG_001 – GET neexistujícího ID vrací 500 NOT_FOUND místo 404	Vysoká	Medium	TC_GET_002, TC_GET_006, TC_GET_012	Opravit mapování výjimky nenalezeného studenta na 404 a zachovat kód NOT_FOUND.
BUG_002 – DELETE vrací 200, ale studenta neodstraní z DB ani GET	Vysoká	Critical	TC_DELETE_001–002, TC_DELETE_008, TC_DELETE_012, TC_DELETE_030–031	Opravit skutečné smazání z DB, kontrolovat transakci; po DELETE ověřit SELECT a GET.
BUG_003 – DELETE neexistujícího ID vrací 500 místo 404	Vysoká	Medium	TC_DELETE_004, TC_DELETE_011, TC_DELETE_026	Pro neexistující ID vracet řízenou chybu 404 místo 500.
BUG_004 – PUT umožňuje změnu na duplicitní e-mail	Střední	High	TC_PUT_010, TC_PUT_049	Vynutit unikátnost e-mailu při PUT stejně jako při POST; otestovat také databázový constraint.
BUG_005 – PUT neočekávaně přepíná isEuCitizen	Vysoká	Critical	TC_PUT_014–016; TC_PUT_027; TC_PUT_029; TC_PUT_034; TC_PUT_049	Opravit částečnou aktualizaci boolean pole; neuvedená pole zachovat, explicitní true/false respektovat.
BUG_006 – příliš velké numberOfFailedStudies způsobí HTTP 500	Střední	Medium	TC_POST_029, TC_POST_031, TC_PUT_042, TC_PUT_055	Validovat rozsah hodnoty před zápisem a vracet 422; ověřit nulový dopad v DB.
BUG_007 – mimo rozsah 32bitového ID vrací 500	Střední	Medium	TC_GET_011, TC_PUT_059, TC_DELETE_010	Ověřit rozsah path parametru před DB dotazem; vracet řízenou validační chybu.